Tecniche Avanzate di Programmazione

LLM

LLM sta per Large Language Models, ossia ChatGPT, Gemini, Copilot, Claude, …

Hanno cambiato il modo in cui risolviamo problemi, ma non sono magia, bensì solo strumenti. Imparare a programmare significa imparare a pensare in modo strutturato per risolvere problemi, abilità che nessun LLM possiede.

Gli LLM hanno molti limiti:

- 'Allucinazioni': dicono cose che sembrano corrette, in modo convincente, ma che sono in realtà completamente false, fabbricate da lui o senza fonti (esempi: invenzione di funzioni o librerie, parametri errati o di librerie passate, citazione di documentazioni inesistenti, ...).
- Knowledge Cutoff: a meno che il modello non si possa connettere ad internet, sarà allenato fino ad una certa data; quindi, non saprà argomenti dopo la data in cui è stato allenato.
- Finestra di contesto limitata: non riescono ad analizzare progetti complessi; un LLM può 'leggere' solo una quantità limitata di testo alla volta.
- Bias e Ripetizione di pattern comuni: vengono allenati con fonti tendenzialmente pubbliche, e può quindi replicarne i difetti, gli errori.

Un LLM agisce quindi come un assistente: può scrivere parti standard, tradurre, spiegare un concetto o un errore, …

Un prompt efficace deve essere specifico, definire il formato di output, dare degli esempi, assegnare un ruolo (esempio: agisci come un professore, ...), iterare (chiedere altre domande dopo la risposta del LLM).

Il codice generato da un LLM non nasce dal nulla; quindi, non possiamo ignorare le possibilità di plagio, di violazione delle licenze, …

LLM non garantisce la privacy, quindi non si deve mandare codice sensibile, come chiavi API, che potrebbero poi essere generate come risposta ad altri utenti o essere lette da ricercatori.

Produzione e distribuzione del software

Storicamente, si è passati da uno sviluppo definito 'in-the small' (singolo programmatore o piccolo team e un'unica codebase; con problemi principalmente tecnici e legati all'ambiente di esecuzione) ad uno definito 'in-the-large' (team distribuiti, milioni di linee di codice; problemi legati anche all'organizzazione tra programmatori, ...).

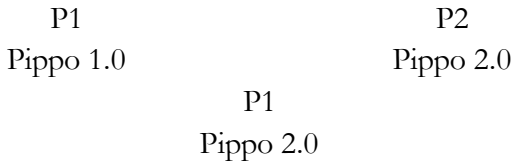
Lo sviluppo 'in-the-large' comprende la collaborazione, molto spesso tramite Git, con cronologia delle modifiche, branch, ossia linee di sviluppo indipendenti che verranno poi uniti nel main tramite merge.

Negli anni '80-'90 si avevano varie architetture hardware (in competizione), per cui un programma doveva essere compilato per ogni combinazione di CPU e sistema operativo, con architetture con ordine (memorizzazione che viene salvata dall'inizio / dalla fine, l'ordine in cui si scrive nei byte si chiama Endianness) dei byte diversi (Big-Endian della Motorola, dove metto il byte più significativo per primo, VS Little-Endian di Intel, dove mette il byte meno significativo per primo), system call con strati di astrazione diversi, …

Perché un programma può funzionare in una macchina ma non in un'altra?

Per l'inconsistenza degli ambienti: dipendenze implicite (librerie o strumenti che lo sviluppatore ha installato sul proprio PC), variabili d'ambiente, percorsi sul filesystem, permessi, DLL Hell, …

'DLL Hell'
Avviene quando una libreria è presente, ma in una versione non compatibile. Il DLL è, in Windows, il file che contiene le librerie che vengono poi linkate quando si esegue un linking dinamico: per risparmiare spazio e per facilità di aggiornamento, un programma non incorpora le librerie da cui dipende (linking statico), ma fa riferimento a file esterni (.dll in Windows, .so in Linux). Quando una libreria veniva aggiornata, l'installer sovrascriveva la versione precedente, e quindi i programmi che avevano la versione passata si trovano ora linkati ad una versione con la quale non sono mai stati testati (quindi potrebbe causare problemi).



(Dopo aver installato la versione Pippo 2.0 per il programma P2, anche P1 avrà la versione Pippo 2.0)

Una soluzione parziale a livello di linguaggio è il 'Virtual Environment': si cerca di isolare le dipendenze di un progetto solo a quel progetto. Prendendo come esempio il virtual environment in Python, è quindi una directory che contiene il programma, la specifica versione dell'interprete Python e tutte le librerie necessarie per il progetto, per inscatolare/isolare il processo. Funziona però solo dal punto di vista delle librerie: non risolve i problemi legati alle variabili d'ambiente, path filesystem, ...

Con la crescita del numero delle dipendenze, un altro problema è la loro gestione: se un programma ha bisogno di tante librerie (es. 50), con versioni diverse, ecc. servono i 'package manager', strumenti che automatizzano il download e l'installazione delle librerie giuste e eventuale aggiornamento. Ogni sistema moderno ne ha uno: con Python si una pip con il file requirements.txt che linka le varie librerie con eventuali vincoli sui numeri di versione, .NET e C# usano .csproj, ...

Se delle librerie dipendono transitivamente ad altre librerie (A dipende da B e B dipende da C), possono esserci incompatibilità in caso di nuove versioni. Per garantire build riproducibili, la soluzione è quindi un 'file di lock', o 'lock file', che 'congela' la versione di ogni pacchetto. Quando il package manager scarica le librerie, oltre ad esse scarica un file di testo 'packages.lock.json' che dichiara le versioni precise delle librerie scaricate. La prossima volta, quindi, non andrà di nuovo a cercare le librerie ma installerà di nuovo quelle nel lock file. Questo vuol dire però che rimarrà 'per sempre' a quella versione.

Problemi dello sviluppo 'in-the-large'
Lo sviluppo in-the-large fa emergere una nuova classe di problemi non solo legati a dove il software gira, ma anche come viene sviluppato, compilato, testato e distribuito. I problemi tecnici, oltre a quelli di processo, sono anche quelli di collaborazione. Lo sviluppo 'collaborativo' è fattibile grazie ai 'version control', 'controlli di versione'. L'esempio principale è Git: puoi lavorare sul file in modo indipendente e poi mettere insieme tutti i file, senza sovrascrivere il lavoro degli altri, e anche in caso di sovrascrittura viene salvata una cronologia per poter 'tornare indietro'.

In Git si possono fare delle modifiche, che possono essere preservate tramite 'commit', ossia una unità di lavoro, che contiene un commento (che descrive le modifiche), autore e data, con un puntatore al commit precedente, creando una cronologia.
Man mano che la codebase cresce, più è possibile che una modifica possa essere rischiosa: una modifica da una parte abbia un impatto su un'altra parte che non c'entra apparentemente niente, in quanro è anche normale non conoscere tutto il codice. Per questo è fondamentale avere dei sistemi automatici di test che vadano a verificare che il test funzioni anche dopo aver fatto la modifica: questi sono chiamati 'unit test', pezzettini di codice che controllano altro codice. Questi test possono essere automatizzati, tramite la pipeline CI/CD.

- Continuous Integration (CI): per ogni commit, un server compila il codice ed esegue l'intera suite di unit test
- Continuous Delivery (CD): se la fase CI ha successo, allora il sistema procede automaticamente a 'impacchettare' l'applicazione (es. creare un'immagine Docker) per il rilascio. Il passaggio in produzione è una decisione manuale.
- Continuous Deployment (CD): come delivery, ma il codice viene rilasciato direttamente in produzione.

Github Actions è uno degli strumenti più popolari per implementare pipeline di CI/CD.

Managed Runtime
I 'Managed Runtime' è un modo per ovviare al problema di dover compilare i programmi per CPU e SO diversi. Sono degli ambienti di esecuzione che vengono messi sopra al livello del sistema operativo e ci offrono un'architettura standard, astraendo le differenze tra le piattaforme. Invece di cambiare il programma per ogni singola architettura, prepariamo questo ambiente unico su cui le applicazioni si appoggiano. Un esempio è la JVM, la Java Virtual Machine: l'idea è quella che quando compilo codice sorgente, al posto di andare a generare il codice macchina di una specifica architettura hardware, vado a specificare un 'codice macchina' che si chiama bytecode che è il linguaggio della JVM, che agisce come un 'processore virtuale' che lo interpreterà. Fanno la 'just in time compilation': prendono un blocco di codice e lo traducono nel codice macchina dell'architettura su cui sta girando. Il problema della JVM è che è ottimizzata solo per java e non per altri linguaggi.
Microsoft si è ispirato a questa idea e ha sviluppato .NET, ma la grossa differenza tra i due è che .NET nasce per essere indipendente dal linguaggio, ossia che possa essere efficiente per tutti i linguaggi, anche se viene 'creato' per usare il C#.

Come funziona?

Codice Sorgente -> Compilatore -> Codice Intermedio (CIL) -(CLR)-> Esecuzione Gestita

Il codice sorgente viene dato in pasto al compilatore, che genera il codice intermedio (CIL, common intermedeate language), ed esegue il codice tramite Common Language Runtime (CLR).
.NET permette facilmente di interfacciare codice scritto in un linguaggio con del codice scritto con un altro linguaggio: quindi posso scrivere un pezzo in C#, un pezzo in F#, ... e possono chiamarsi a vicenda, passarsi oggetti uno con l'altro, ecc.

I servizi principali del CLR includono:

- Compilazione Just-in-Time (JIT): il CLR non interpreta il codice intermedio (CIL), ma lo compila in codice nativo la prima volta che un metodo viene chiamato; potrebbe ricompilarlo dopo aver

fatto profiling (osserva cosa fa l'applicazione, il sistema si accorge di quali sono le parti più usate e le compila meglio per rendere il codice più efficiente).

- Gestione della memoria (Garbage Collector): il sistema tiene traccia dei riferimenti agli oggetti, e gli oggetti non più raggiungibili vengono rilasciati automaticamente. Questo porta a varie complicanze in caso ad esempio di multithrading.
- Verifica dei tipi (Type Safety): il CLR applica il Common Type System (CTS), che permette a linguaggi diversi di interagire senza problemi.
- Gestione delle eccezioni: deve essere anche cross-language.

C#

Linguaggio progettato fin dal principio per essere il linguaggio primario dell'ecosistema .NET; è un linguaggio Object Oriented, fortemente tipato, moderno e sempre in evoluzione, introduce nuove funzionalità (esempi: LINQ, async/await per la programmazione asincrona), …
Il primo .NET era '.NET Framework' che era esclusivamente per Windows. Microsoft ha poi creato .NET Core: open-source, cross-platform e ad alte prestazioni. Infine, è diventato solo .NET, e ha le stesse caratteristiche del Core.
Con C# si possono creare siti web, applicazioni desktop e mobile, videogiochi (Unity), …
C#, oltre a poter interfacciarsi con il codice nativo (rischioso) come Java, ha anche la possibilità di usare dei costrutti che sono stati marcati come 'unsafe'. Come in codice nativo, se usiamo puntatori ecc si può fare casino quindi questi blocchi non si usano!!!

Virtualizzazione e Containerizzazione

Per risolvere il problema del 'it works on my machine', si può usare una Macchina Virtuale (Virtual Machine), che permette di emulare una macchina con il suo sistema hardware, il suo sistema operativo, le sue librerie, … Su questa macchina emulata è possibile installare un sistema operativo completo, chiamato OS Guest, esattamente come si farebbe su un computer fisico.

Il componente software principale si chiama Hypervisor (o Virtual Machine Monitor, VMM) che si interpone tra l'hardware fisico e le Virtual Machine, e ci sono due modi:

- Bare-Metal: installato direttamente nell'hardware, come un Sistema Operativo, un'interfaccia che ti permette di creare e configurare Virtual Machine (usati di solito per varie virtual machine in computer cloud, non su pc 'normali')
- Hosted: gira come un'applicazione su un SO preesistente, l'OS Host

Nei sistemi operativi è necessario che la CPU supporti due modalità di esecuzione che sono quello 'utente' e quello 'kernel'. Il kernel è il nucleo del sistema operativo e gestisce l'hardware; sopra di esso abbiamo delle librerie e poi ci sono le varie applicazioni. Quando un'applicazione e segue istruzioni che non devono interagire con l'hardware (somma, leggere/scrivere dalla RAM, ...), l'istruzione viene eseguita direttamente dall'hardware (la CPU esegue le somme, moltiplicazioni, salti, confronti, ...). Quando però si deve parlare con un disco (es: leggere un file), l'applicazione non può parlargli direttamente, perché se potesse parlargli sempre allora potrebbe leggere e scrivere qualsiasi cosa. Queste istruzioni sono dette 'privilegiate', e possono essere eseguite solo quando la CPU è in modalità kernel. In questi casi l'applicazione fa una 'system call', invoca il sistema operativo, quindi passa alla modalità kernel e il codice del kernel tramite un driver andrà a parlare ad esempio con il disco. Quando questa istruzione finisce di essere eseguita, la modalità di esecuzione della CPU tornerà ad essere user.

In una Virtual Machine, il suo kernel pensa di star girando in modalità kernel, ma in realtà sta girando in modalità utente. Quando una app fa una system call genera una trap, si passa in modalità kernel della VM, e il VMM (Virtual Machine Monitor) riconosce che questa trap era una richiesta di system call da parte di una app della VM, e reindirizza la system call al kernel della VO. Ma se il kernel vuole ad esempio eseguire una lettura da disco, allora quando prova si genera un'altra trap che lo porta nel kernel 'originale' del SO Host. A questo punto il VMM si accorgerà che il kernel della VM vuole parlare con la unità disco, e via software gli emuliamo il dialogo con l'unità disco. Di solito quando si crea una VM il suo disco è un file nella macchina Host. Quindi il VMM emula, dà l'illusione di parlare con l'hardware, con il disco, ... Per questo quando si virtualizza il software va più lento, anche se ci sono tools che possono migliorare le interazioni ecc.
I diver girano con gli stessi privilegi del kernel, quindi se c'è un bug nei driver è un grosso problema. Ad oggi ci sono liste con i driver che il pc sa di non poter scaricare, ma un tempo venivano scaricati comunque (perché effettivamente erano legittimi e firmati da windows ma avevano vulnerabilità che venivano sfruttate da attaccanti).

I vantaggi della virtual machine sono:

- Con 'poche' machine hardware potenti, si può far girare sopra tante virtual machine
- Ogni macchina virtuale esegue il suo sistema operativo ed è come avere macchine diverse

Gli svantaggi della virtual machine:

- Grande spreco di risorse (ogni VM ha le sue librerie, copia del suo sistema operativo, …)
- Lunghi tempi di avvio

Le virtual machine piacciono perché riescono a isolare, però hanno il difetto che si portano dietro l'intero sistema operativo, ... sprecando risorse.
Una soluzione a questo sono i container, ossia un modo per creare un isolamento analogo a quello delle virtual machine ma in modo molto più leggero. Mentre nelle VM abbiamo l'hardware, il sistema operativo Host, l'hardware finto, il sistema operativo Guest1, le applicazioni, ... La differenza è che nel container i sistemi operativi sono diversi ma il kernel è quello della nostra macchina OS Host; quindi, SO1 non sa che esiste sistema SO2 o SO3. Questo vuol dire che non possiamo fare tutte le cose che si possono fare con una VM, perché si hanno più limitazioni hardware. Il vantaggio è che, quando avvio il container non devo are il boot della macchina o inizializzare un sistema operativo, ma devo solo far partire un processo sullo stesso kernel che sta già girando, ma che avrà dei vincoli (pensa di essere l'unico processo del sistema, non vede quelli che girano in altri container o al di fuori dei container). Partono in pochi secondi, invece che in minuti come la VM, non sprechiamo spazio avendo varie copie del SO, ma abbiamo un unico SO. Ogni container vede il suo file system, quindi sono isolati come Virtual Machine. Però se si avesse un kernel Linux, avremmo solo container Linux; se si avesse un kernel Windows si avrebbero solo container Windows, ... Pero se si scarica Docker su Windows, di default ha i container Linux, quindi dietro le quinte crea una Virtual Machine in cui fa girare i container.
Possiamo combinare l'idea di Managed Runtime come .NET alla containerizzazione. Quindi possiamo sviluppare una nostra applicazione su .NET e poi impacchettarlo dentro ad un container, all'immagine di un container. In questo modo possiamo fare l'upload di questo pacchetto su un server pubblico come Docker Hub, e chiunque nel mondo può lanciare la nostra applicazione. Docker caricherà questa immagine e la farà girare su quella macchina e dentro al container ci saranno tutte le librerie, tutti i file, il file system che serve per far girare l'applicazione e sarà uguale per tutti.

I container sono nati per essere leggeri e senza uno stato, ovvero che quando termino di runnare il container sparisce tutto il file system, il processo, ... Quindi se un container volesse salvare dei dati si dovrebbero creare dei volumi.
Quando le applicazioni crescono e diventano sistemi complessi, composti da molti container (microservizi), vengono gestiti da un orchestratore che si occupa di scheduling, self-healing (se un container crasha ne avvia uno nuovo), scalabilità (aumentare o diminuire il numero di container in base al carico di lavoro), load balancing. 'Kubernetes' è forse il più famoso orchestratore open-source.

Lo sviluppo di software moderno è un ecosistema di strumenti e pratiche e ognuno serve per affrontare problemi a livelli diversi:

- Collaborativo: Git e metodologie che permettono ai team di lavorare in modo efficace
- Ambiente/Riproducibilità: container che permette di avere lo stesso ambiente di esecuzione quasi ovunque
- Portabilità: JVM o CLR che astraggono il sistema operativo, permettendo al codice di girare ovunque
- Qualità: l'unit testing fornisce una rete di sicurezza contro le regressioni, garantisce affidabilità del software e facilità per refactoring ed esperimenti
- Automazione: CI/CD automatizzano build e controlli di qualità, garantendo che solo codice funzionante venga integrato

---

Git

Introduzione

Un Sistema di Controllo di Versione (VCS) è uno strumento per gestire l'evoluzione di un file di testo, in particolare sorgenti (codice, che però non controlla sintatticamente ma tratta appunto come normali file di testo). Permette a più persone di lavorare su dei file condivisi, sullo stesso progetto, senza sovrascrivere il lavoro altrui. I sistemi più moderni permettono anche di lavorare su nuove funzionalità in isolamento, ossia che se ho un progetto che funziona, compila, ecc. e voglio provare ad implementare una nuova funzionalità, si può creare un branch, un ramo, da cui si può iniziare a sviluppare questa nuova funzionalità/refactoring del codice. Il branch principale è sempre lì e continua ad esistere e a funzionare come ha sempre funzionato, e durante lo sviluppo ipotetici bug sarebbero presenti solo nel branch della funzione che si sta implementando. Quando la nuova feature è stata implementata si può aggiungere al branch principale con merge, oppure eliminarlo senza modificare il resto del progetto. Tiene traccia di ogni modifica, da parte di chi e quando ha fatto qualcosa: fungono da 'macchina del tempo', ed è facile tornare/controllare ad una versione precedente.
I Sistemi di Controllo di Versione VCS si dividono principalmente in due categorie:

- Centralizzato (CVCS): tutta la cronologia è in un unico server centrale. Gli sviluppatori fanno il 'checkout' della versione più recente, la scaricano in locale e ci lavorano (in locale), poi 'committano' le loro modifiche sul server centrale. Tutti i sistemi più vecchi sono centralizzati; i sistemi più antichi erano molto limitati, e quando uno sviluppatore prendeva un file dal server, impediva a tutti gli altri di lavorare in quel file, in quanto avevano un lock su quel file. I sistemi un po' più moderni (da CVS in poi) permettono a tutti di avere sulla propria macchina una copia dei file, fare le modifiche e poi mandarli al server. Chiaramente, possono esserci problemi in caso in cui la versione che viene scaricata in locale cambia rispetto a quella nel server, magari per delle modifiche da parte di altri sviluppatori: se accade in parti diverse del file o in file diversi, si fa

merge e semplicemente vengono uploadate sul server entrambe le modifiche (mia e dell'altro sviluppatore); se la modifica fosse nelle stesse linee del file, il sistema di controllo di versione non saprebbe qual è quella giusta, e l'unica cosa che può fare sarebbe segnalare un conflitto.

- Distribuito (DVCS): ogni sviluppatore quando fa un'operazione di clone non prende la versione attuale dei file, ma prende tutta la cronologia dei file. Vengono salvati in locale in dei repository; può funzionare anche da backup, e questo rende possibile lavorare offline. In teoria è completamente distribuita (quindi quando faccio delle modifiche potrei mandarle a due sviluppatori, che possono sincronizzarsi e mandarmi una terza versione, ...) ma in pratica si tende ad avere comunque un server centrale (esempio GitHub) che hosta i repository, e gli sviluppatori clonano da quello e mandano le modifiche su quello.

CVS (Concurrent Version System) è il primo Sistema di Controllo di Versioni (VCS) moderno a diventare popolare. É il primo sistema che permette di lavorare contemporaneamente sugli stessi file: lo sviluppatore prende il file, ci lavora sulla sua macchina e poi manda le modifiche al server. È centralizzato, ma ha grosse limitazioni come non poter rinominare i file o spostarli. Ormai è solo di interesse storico e ha gettato le basi per Subversion.
SVN (Subversion) è nato come 'CVS fatto bene', e risolve molti problemi: è centralizzato, con commit atomici garantiscono la consistenza (che CVS non aveva, funziona come il journal in caso di problemi di connessione/corrente, ecc.), possibilità di poter rinominare e spostare file/directory, ma il branching rimane ancora poco flessibile.
HG (Mercurial) è uno dei primi sistemi distribuiti, open source, nato e noto per la sua semplicità d'uso. É stato contendente di Git per qualche anno, è sviluppato in Python. Quasi tutti usano Git, ma anche Mercurial è valido, ma è un po' meno efficiente in caso di progetti grandi.

Origini di Git

Caratteristica	CVS	Subversion (SVN)	Mercurial (Hg)	**Git**
Modello	Centralizzato	Centralizzato	Distribuito	**Distribuito**
Commit Atomici	No	Sì	Sì	**Sì**
Velocità	Lento	Moderato	Veloce	**Molto Veloce**
Branching	Difficile	Migliorato	Facile e Veloce	**Facile e Veloce**
Offline	No	No	Sì	**Sì**
Curva di Apprendimento	Facile	Facile	Media	*Media/Difficile*
Popolarità	Obsoleto	In declino	Di nicchia	**Standard di fatto**

Il kernel Linux veniva sviluppato usando BitKeeper, di cui avevano la licenza gratuita. Nel 2005 la licenza gratuita viene revocata (per casini) e quindi il team di sviluppo si trova improvvisamente senza un Sistema di Controllo di Versione per il kernel. Linus Torvalds decide di creare un tool da zero, con obbiettivi principali la velocità, design semplice, branching efficiente, distribuito e integrità. Dopo circa 10 giorni aveva già una prima versione funzionante.

Dopo l'abbandono di BitKeeper da parte del kernel di Linux, e la creazione di Git, molti altri progetti seguirono l'esempio. Git è open-source e gratuito, e rese molto difficile per BitKeeper competere. Nel 2016 per cercare di riguadagnare rilevanza, diventa open-source, ma niente lol.
Ormai quasi tutti usano Git. É l'unico con un concetto di Staging Area, che permette di selezionare quali modifiche includere nel prossimo commit. Tutti gli altri sistemi quando si fa commit dopo aver eseguito delle modifiche, prendono tutti i file con tutte le modifiche e vengono caricate sul server; nel caso di Git, dobbiamo dire noi quali file (o addirittura solo pezzi di file) vogliamo includere nel commit.

Configurazione: La configurazione di Git è tramite file di tipo .INI con:

- Sezioni dichiarate fra quadre, come [user] o [core]
- Associazioni chiavi/valori, come name = Giovanni Lagorio
- Commenti monolinea che iniziano con # o ;

Sono file di testo che si possono anche scrivere a mano in quanro non particolarmente complicati, poi ci sono dei comandi git per andare invece da riga di comando a impostare le configurazioni.
Il comando **git config** permette di visualizzare e impostare le variabili di configurazione.
Git gestisce le opzioni di configurazione in maniera gerarchica:

- Sistema (su sistemi Unix /etc/Gitconfig): modificato raramente, sono le impostazioni applicate a tutti gli utenti
- Globale (nella cartella home dell'utente): applicato a tutti i repository dell'utente
- Locale: si applica solo ad un repository specifica

Alcune configurazioni iniziali sono quelle di specificare il **user.name** e la **user.mail**, e tipicamente si fa a livello globale. Altre configurazioni che di solito sono globali sono **core.editor**, che definisce quale editor far lanciare a Git per inserire un commento in un commit; **init.defaultBranch = main**, di default per Git è master ma GitHub usa main dal 2020 e quindi si può cambiare.

git config --global user.name "…"
git config --global user.email "…@…"

Per vedere il valore di una configurazione, esempio user.name, scrivo **git config user.name**.
Per vederle tutte, compreso dove vengono impostate, scrivo **git config --list --show-origin**. Se non scrivo --show-origin mostra solo la lista delle configurazioni.

Repository: Per creare una directory e farla diventare un repository scrivo:

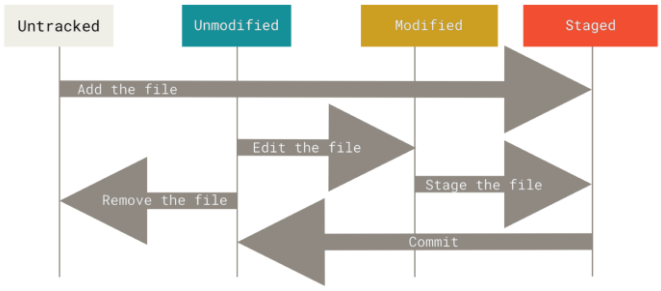
mkdir TapGitDemo && cd TapGitDemo && git init

Il comando **git init** crea il repository. Di default ls non elenca i file che iniziano con il punto. Nel momento in cui viene creata un repository, viene creata una cartella .git, dove finiscono tutte le robe di git. Quindi un repository Git è una cartella qualsiasi che ha un .git in cui finisce tutta la roba di git: se si cancella la sottodirectory .git, la cartella smette di essere un repository.
All'interno di un repository, un file può essere in due stati diversi:

- Tracked: git gestisce il suo stato
- Untracked: git non ne sa nulla (non gestisce il suo stato)

Quando aggiungo un file al repository, questo file può essere in tre stati diversi:

- Unmodified/committed: non modificato rispetto alla versione precedente, salvato in repository e non più modificato
- Modified: modificato rispetto alla versione conosciuta da Git
- Staged: modificato e marcato per essere incluso nel prossimo commit



All'inizio il file è untracked; con un'operazione di add aggiungiamo il file e va dentro la Staging Area, e questo vuol dire che è pronto per essere incluso nel prossimo commit. Se noi andiamo a modificare il file diventa modified. Nel momento in cui facciamo il commit smette di essere modified, cioè le modifiche che avevamo eseguito diventano la versione corrente del file (che viene poi recuperato dal server,ecc.).

Il comando **git status** elenca lo stato in cui sono i file e dice anche che comandi fare per cambiare stato.

```
gio / tmp tap_2025_10_01 TapGitDemo (e) py3 main echo ciao mondo > pippero
gio / tmp tap_2025_10_01 TapGitDemo (e) py3 main* git status
On branch main

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        pippero

nothing added to commit but untracked files present (use "git add" to track)
gio / tmp tap_2025_10_01 TapGitDemo (e) py3 main* git add pippero
gio / tmp tap_2025_10_01 TapGitDemo (e) py3 main git status
On branch main

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   pippero
```

```
gio / tmp tap_2025_10_01 TapGitDemo (e) py3 main git rm --cached pippero
rm 'pippero'
gio / tmp tap_2025_10_01 TapGitDemo (e) py3 main* git status
On branch main

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        pippero

nothing added to commit but untracked files present (use "git add" to track)
```

Cache e Staging Area sono spesso sinonimi per ragioni storiche; quindi, **--cached** vuol dire che lo tolgo solo dalla Staging Area, ossia non far parte del commit.

Per fare un commit scrivo **git commit**. Questo aprirà un editor per commentare il commit; il commit è un messaggio che serve per le altre persone con cui si lavora per far sapere che cosa si ha modificato.

Per scrivere il messaggio direttamente su linea di comando e non aprire l'editor, posso scrivere **git commit -m "…"**.

Se modifico un file, posso usare **git diff** per vedere le differenze che sono state apportate rispetto alla versione precedente; con **git diff --staged** posso vedere le differenze tra Staging Area e l'ultimo commit; con **git diff <hash1> <hash2>** si può vedere la differenza tra due commit tramite i loro hash (con cui vengono identificati, spiegato meglio tra un paio di righe lol).

Il concetto di Staging Area (per ragioni storiche chiamata anche index o cache) è una delle caratteristiche distintive di Git. È una zona intermedia dove preparare il prossimo commit, dove inserire le modifiche che vogliamo registrare permanentemente con un commit. Può dividere in modo più granulare le modifiche che fa, ad esempio modificando solo certi file specifici.

Se nel commit voglio includere tutti i file che ho modificato, invece che fare **git add** con tutti i file, si può mettere **-am**. Posso anche ad esempio aggiungere solo un file alla Staging Area, anche se ne ho modificati due, e nel commit verrà inserito solo uno, mentre l'altro rimane nello stato modified.

Un commit dal punto di vista logico crea una sorta di fotografia dello stato del repository; quindi, in qualsiasi momento in futuro potremmo tornare allo stesso esatto stato dei file in cui si era in quello specifico commit. Un commit è composto da un autore, tutti i file che fanno parte del repository e un messaggio/commento che descrive i cambiamenti di quel commit.
Un commit viene identificato da un hash SHA-1. Per vedere i commit (i loro hash) possiamo usare il comando **git log** o **git log –oneline** (che accorcia un po'). Quando si vuole fare rifermento ad un commit, si può fare tramite codice hash, ma non serve specificare tutto l'hash, ma un prefisso che non sia ambiguo (i primi valori, esempio: 0e7... l'hash è più lungo ma uso solo i primi valori).
Se dopo aver eseguito un commit ci rendiamo conto di aver fatto un errore, **git commit --amend** permette di modificare l'ultimo commit più recente. Oltre a cambiare il messaggio, cambia anche l'hash, perché quest'ultimo è un'impronta che riassume tutte le informazioni che sono dentro al commit; quindi, se cambio il commit cambierà anche l'hash. Se il commit che ho cambiato ce l'avevo solo io non ci sono problemi; se però quel commit era diventato pubblico, e un'altra persona l'aveva clonato, gli rovino il prossimo push perchè il suo commit non avrà più un commit a cui puntare!!!
Il commit attuale si chiama head, e punta qual è l'ultimo commit (ossia il commit dove sto facendo le modifiche). Se volessi tornare indietro a dei commit passati, posso puntare head al commit che voglio ma questo vuol dire che i commit successivi verranno eliminati in quanto non avranno più un padre a cui puntare.
In molti casi, ci sono file o cartelle che non ha senso mettere nel repository, come file binari, file di log, file di configurazione dell'editor (come .vscode), dati sensibili, chiavi, password, … Usiamo quindi il file **.gitignore** che ti fa specificare i file che non vuoi inserire. Il file **.gitignore** finisce invece nel repository. Su GitHub ci sono una serie di file .gitignore per i vari linguaggi che vengono utilizzati: se uso ad esempio .NET, posso scaricarmi il file .gitignore fatto per .NET che contiene già tutte le convenzioni di .NET che non vanno messe sul repository.

'Porcelain' e 'Plumbing'
Ci sono una serie di comandi di Git molto di basso livello, utilizzati principalmente internamente da Git e che noi usiamo molto raramente, e poi ci sono una serie di comandi ad altro livello, quelli che usiamo tutti i giorni. I comandi 'plumbing' sono quelli di basso livello, mentre i 'porcelain' sono quelli ad alto livello.
I commit sono identificati da un hash; possiamo dare un'occhiata dentro a .git/objects per vedere cosa c'è. Dentro a un repository git ci possono essere tantissimi file; se tutti i file finissero in un'unica cartella ci sarebbero problemi. Quindi viene tutto diviso in tante cartelle in modo uniforme: utilizzando l'impronta hash, i primi due caratteri vengono utilizzati per la cartella e il resto per il nome del file. Siccome per la proprietà delle funzioni hash quei bit sono distribuiti in modo uniforme, abbiamo un modo 'furbo' di distribuire i file in queste sottocartelle.
I file che vengono messi dentro a .git vengono compressi, e quindi se volessi vederli con cat vedrei la versione compressa di qualcos'altro. Se voglio vedere la versione decompressa, posso usare **git cat-file –p <hash>**, ossia un comando 'plumbing'. Questo ritorna un file ASCII che ritorna l'albero, il parent (a meno che non sia il primo commit in assoluto della directory), l'autore, il committer. Un albero è anche lui un file in .git/objects. Un commit è una fotografia del contenuto del repository, quindi quando noi creiamo un commit c'è un oggetto che lui chiama albero e sotto ci sono i file che andiamo a includere nel commit, e al loro interno contengono dei blob, ossia il contenuto del file. I file che mettiamo sono oggetti, i commit sono oggetti, i commit sono oggetti che fanno riferimento ad altri oggetti che sono alberi, che

potete pensare come una sorta di elenco di file/directory. Un commit può avere uno o più parent: quando si fa il merge di più branch si avrà un commit con più parent.

.git/head è un file che punta all'ultimo commit su cui stiamo lavorando. **Git status** ritorna lo stato del file e ritorna cosa fare per modificare il suo stato (esempio: file modificati che non sono ancora nel commit ti suggerirà di fare add al commit, ...). Se faccio **git commit –a** aggiunge al commit in automatico tutti i file modificati che git conosce già (in quanto già tracciati da git). Con **cat .git/refs/head/main** ritorna l'hash dell'ultimo commit.
Ogni volta che faccio un commit creo un grafo, con il primo commit come unico senza parent; tutti gli altri ne hanno almeno uno, se si fa merge il commit avrà più parent.
Tree è un oggetto Git che tiene traccia di quali sono i file che fanno parte del repository: l'hash segna il suo contenuto (in blob).

Branching
Un branch è una linea di sviluppo e per Git il branch è semplicemente un puntatore ad un commit. Head è il commit su cui stiamo lavorando. Nella forma più comune (nome di un branch), head punta al branch e il branch punta al commit. Esiste anche la possibilità di puntare head direttamente su un commit (detached head). Il problema principale di head che punta direttamente al commit è che si rischia di perdere pezzi, perchè il branch ricorda dove siamo arrivati (cronologia), mentre se head è staccata potresti 'perderti'; in alcuni casi ha senso farlo (esempio: voglio verificare che una verisone specifica del mio lavoro passi test, quindi dici 'lanci i test su questo commit e ti do l'hash'; allora il sistema di continuous integration prenderà quella versione specifica con la head staccata, visto che tanto non andrà ad aggiungere commit), ma principalmente noi non la stacchiamo dal branch. Per 'staccare' la testa, si usa **git checkout**, e per 'riattaccarla' si usa **git switch main** (si può usare di nuovo anche git checkout ma è teoricamente obsoleto).
Normalmente quando si fa commit viene creato un nuovo oggetto commit che avrà il suo hash; gli verrà messo come parent la head attuale, dopodichè il puntatore del main verrà aggiornato in modo da puntare all'ultimo commit del branch. Il branch main, man mano che aggiungiamo commit, cresce, e ogni volta che mettiamo un commit il contenuto del file main verrà aggiornato rispetto all'ultimo commit che abbiamo. Se invece ho la testa staccata e faccio un commit, head viene aggiornato per puntare al nuovo commit (con un certo hash, ecc.). Se lavoriamo con la testa staccata, main è uno dei tanti branch nel repository. Se io ricordo esattamente l'hash dell commit fatto con la head staccata, posso anche recuperarlo, ma è difficile (a meno che non dai un nome al branch per seguire entrambi).
Comandi:

- **git branch** ritorna una lista di tutti i branch e indica con un * in quale siamo.
- **git branch <nome-branch>** crea un branch.
- **git branch -d <nome-branch>** elimina un branch.
- **git switch <nome-branch>**per spostarmi di branch.
- **git switch -c <nome-branch>** crea un nuovo branch e ci si sposta subito.

Creando vari branch, con varie etichette, sullo stesso commit, ci permetterà di andare avanti in un 'ramo' senza che gli altri si rompano in qualche modo.
Come abbiamo detto prima, **git chekout** può essere usato per:

- Cambiare branch (adesso **switch**)
- Scartare le modifiche di file (adesso **restore**)

- Esplorare la cronologia, recuperando specifiche versioni di file

Ad oggi si cerca di utilizzare le alternative nuove, ma un tempo checkout si usava per tutto.

Esempio:

Creiamo un 'hello world' in C# con .NET:

dotnet new console –n HelloTAP

Se faccio cd HelloTAP e faccio **dotnet run**, ritornerà la scritta 'Hello, World!'. Dopo averlo runnato, nella cartella troveremo molti file temporanei di .NET, generati in automatico, e non avrebbe senso condividerli nel repository. Quindi nel file **.gitignore** specifichiamo che tutti i file che sono ricreabili a partire dal sorgente sono inutili da mettere nel repository. Si possono trovare i file .gitignore su github ecc. Quindi se nel commit aggiungo i file di progetto (Program.cs e HelloTAP.csproj), e poi aggiungo il .gitignore, quando faccio git status non mi suggerirà di aggiungere gli altri file (bin, obj).

Se volessimo 'sviluppare una nuova feature', per evitare di distruggere la versione attuale che sappiamo funzioni, creiamo un branch su cui lavoreremo su questa nuova feature.

git switch –c feature/user-greeting

Modifichiamo il programma in modo che chieda il nome dell'utente per ritornare 'Ciao, <nome>!'.

Il Codice è:

```
Console.WriteLine("Enter your name:");
Var name = Console.ReadLine();
Console.WriteLine($"Hello, {name}!");
```

Quindi se switcho su main, il programma continuerà a ritornare 'Hello, World!', mentre nel branch della nuova feature avremo un output diverso.

Quando vorrò unire le modifiche che ho fatto in un branch con il branch principale, utilizzerò la funzione di merge. Per farlo vado nel branch principale main e scrivo **git merge <nome-branch>**.

```
giuliaponassi@MBP-di-Giulia HelloTAP % git merge feature/user-greeting
Updating 9096d9f..9c15ece
Fast-forward
 HelloTAP/Program.cs | 5 ++++-
 1 file changed, 4 insertions(+), 1 deletion(-)
giuliaponassi@MBP-di-Giulia HelloTAP % git log --oneline
9c15ece (HEAD -> main, feature/user-greeting) greeting utente
9096d9f prova ignore
622fcee prova
```

Ci sono due tipi di **merge**:

- Fast forward: abbiamo un branch main e un altro branch che vogliamo attaccare, con vari commit; il branch di partenza però non è stato toccato e quindi abbiamo modifiche solo nel secondo branch dove abbiamo aggiunto delle modifiche. Quindi in questo caso, viene spostato in avanti il puntatore di main all'ultimo commit del branch 'feature'. (Caso più facile possibile)
- Three-way merge: se entrambi i branch hanno nuovi commit, Git crea un 'merge commit' con due parent: l'ultimo commit di main e l'ultimo commit della feature, per preservare entrambe le cronologie. Questo commit andrà ad integrare le modifiche fatte nei due branch. Se abbiamo modificato lo stesso file ma in punti diversi, git riesce ad unire le modifiche; però, se nei branch sono state modificate le stesse linee di codice, git non sa quale scegliere. Si chiama 'three-way' perchè Git va a recuperare nei due branch qual è l'antenato comune, e quindi considera tre commit: i due finali dei branch e il loro più recente antenato comune, ovvero il commit da cui i due hanno iniziato a divergere.

Possiamo vedere la situazione dopo il merge con **git log --graph --oneline --all**, per vedere il grafo.

Esempio in cui le cose vanno male:

Per andare 'indietro nel tempo' ci sono due possibilità:

- Spostare head su un commit precedente, spostare il branch all'indietro, per 'dimenticare' le ultime modifiche e tornare a puntare ad una versione più vecchia. Per farlo si usa **reset**.
- Usando reverse, crea un nuovo commit con le modifiche inverse a quello che c'era prima. Questo non cambia la storia, quindi se ho già condiviso il grafo con altre persone, non posso cancellare in quel modo (meglio reset).

Quindi se scrivo **git reset --hard HEAD~1**, head si sposta al commit prima del fast-forward.
Se provo a modificare il file e poi a rifare merge, ci darà errore.

```
giuliaponassi@MBP-di-Giulia HelloTAP % git merge feature/user-greeting
Auto-merging HelloTAP/Program.cs
CONFLICT (content): Merge conflict in HelloTAP/Program.cs
Automatic merge failed; fix conflicts and then commit the result.
giuliaponassi@MBP-di-Giulia HelloTAP % git status
On branch main
You have unmerged paths.
  (fix conflicts and run "git commit")
  (use "git merge --abort" to abort the merge)

Unmerged paths:
  (use "git add <file>..." to mark resolution)
        both modified:   Program.cs

<<<<<<< HEAD
<feff>Console.WriteLine("Hello, Zena!");
=======
<feff>Console.WriteLine("Enter your name:");
var name = Console.ReadLine();
Console.WriteLine($"Hello, {name}!");

>>>>>>> feature/user-greeting
```

In questo caso i due file sono molto piccoli, però fosse un file molto più lungo potrei avere del contenuto del file che andava bene e un altro pezzo che è in conflitto, segnato con -. Per risolvere posso ad esempio lasciare entrambe le cose, modificare l'output, ecc. Però appunto è una cosa che deve scegliere lo sviluppatore, non può farlo git.

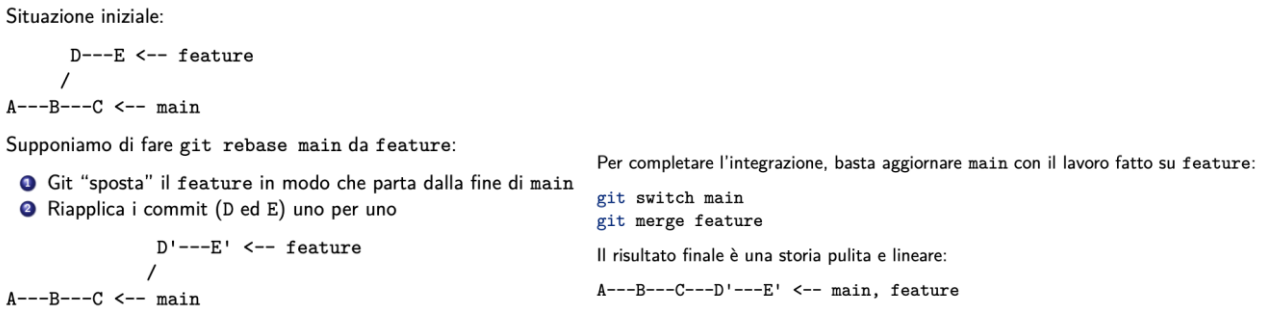
```
giuliaponassi@MBP-di-Giulia HelloTAP % git log --oneline --graph --all
*   229dc1b (HEAD -> main) merge
|\
| * 9c15ece (feature/user-greeting) greeting utente
* | 1e697f6 w la focaccia
|/
* 9096d9f prova ignore
* 622fcee prova
```

Quindi questo nuovo commit ha due parents! Infatti, se controllo con **git cat-file –p <hash>**, vedrò che l'ultimo commit ha due parent.
Come si può immaginare, se ogni volta che si fa un branch sc'è questa situazione, il grafico dei commit può diventare abbastanza complicato. Alcuni preferiscono 'tenerlo complicato' così che possano tenere tutta la storia/cronologia. Invece, con il **rebase**, al posto di fare merge, quando avete due linee separate, si prendono i commit fatti su un certo branch e applicarli uno dopo l'altro su un altro branch. Ovviamente l'identità del commit cambia (l'hash cambia), quindi stiamo 'riscrivendo la storia'. Quindi se il branch è stato fatto 'in privato', abbiamo modificato, testato, ecc. e vogliamo aggiungerlo al branch (SEMPRE SOLO SE NESSUNO HA VISTO IL BRANCH CHE VOGLIAMO AGGIUNGERE), posso applicare i commit al branch principale come se li avessi fatti direttamente lì però sono già testati quindi so già che vanno bene. Questo rende possibile preservare una storia più pulita e lineare.

In dettaglio, cosa fa il rebase?

Come con merge, trova l'antenato comune. Se voglio fare il rabase di b1 in b2, parte salvando tutti i commit fatti in b1 dopo l'antenato comune, sposta b1 per pentoare allo stesso commit di b2, poi prende i commit di b1 messi da parte e li riapplica uno alla volta, in ordine. Se funziona tutto, alla fine avremo b1 che è semplicemente un'estensione di b2, e a quel punto si può fare un merge forward e riportare il puntatore davanti. Se però ci sono conflitti mentre prova ad aggiungere i vari commit, allora ci sarà un errore, che possiamo risolvere, fare add e poi **git rebase --continue**. Se volessimo tornare indietro, si può fare **git rebase --abort**.

Quindi, il merge viene fatto tutto insieme e poi se ci sono conflitti devono essere risolti 'tutti insieme', e una volta risolto ci sarà un unico commit, con due parent, che corrisponde all'operazione di merge.
Il rebase, invece, è un'operazione che applica un commit dopo l'altro e se tutto va bene, quando si ha finito, si può fare un merge fast-forward, risultando quindi un branch lineare.



Non ha molto senso usare rebase se si è già condiviso con altri il grafo.
Non c'è un modo giusto o sbagliato di utilizzare branch. Ci sono delle cosidette strategie di branching, ce ne sono svariate, come Git Flow e GitHub Flow. In generale si ha sempre il main che è il branch principale che è sempre stabile, compila, passa i test, ... Abbiamo un branch di sviluppo dove abbiamo le cose in beta test, e poi possiamo avere tanti feature branch per quando voglio fare esperimenti. Se vanno bene, dopo averli testati, li vado ad includere nel main, ecc. Ma sono solo convenzioni, non c'è un metodo specifico.

É possibile mettere dei tag (etichette, es: release 1.0, ...) a dei commit, usando il comando **git tag <nome tag>** per un'etichetta lightweight (puntatori ad un commit) oppure **git tag <nome tag> -m 'messaggio del tag'** per tag annotated, che sono oggetti completi in Git, con autore, data, ecc. E vengono usati principalmente per le release pubbliche.
Se mentre sto lavorando mi rendo conto ce c'è un problema su un altro branch, ma non sono ancora pronto a fare il commit dall'altra parte, esiste il comando **git stash** che ti permette di accontonare il contenuto della working directory, fare dell'altro e poi riprendere le cose salvate prima. **Quando si fa git stash push –m 'messaggio',** vengono 'impilate' le modifiche di cui non ho fatto il commit, sia staged che unstaged, in una pila stash locale (non condivisa con altri), lasciando la directory di lavoro pulita. Per riprendere le modifiche dalla pila, si fa **git stash pop**. Se il branch è privato si può anche fare il commit comunque, fare lo switch e poi tornare nel branch e eventualmente fare **amend** per fixarlo; **git stash list** mostra la pila delle modifiche impilate.
Con rebase è anche possibile fare lo 'squash' dei commit: se si è fatto un branch con molti commit granulari, ma si preferisse raggruppare le modifiche in due/tre commit, si possono appunto raggruppare i commit in commit più 'grossi', per avere una storia più 'compatta'.

Git: sviluppo collaborativo
Finora era tutto in locale, per parlare con il 'resto del mondo' si usano i remote. Un remote è un altro repository git; può essere una cartella nello stesso file system, ma tipicamente è ospitato in un'altra macchina. Servono sia per collaborazione, perchè permettono più persone di lavorare sullo stesso progetto, che come backup: all'interno di un repository Git c'è tutta la cronologia, quindi clonando il repository, ho una copia completa di tutto quello che è successo.
In teoria, Git è un sistema completamente distribuito, ma in pratica si usano server come Github che fanno da punto di riferimento.

Per connettere un repository a uno o più remote, si clona un repository sulla nostra macchina, e tra i remote troveremo un remote chiamato **origin**, ossia il repository di cui abbiamo fatto la copia con **git clone**. Tramite GitHub, si può usare **git remote add origin <url>**. Con **git remote –v** vedo la lista dei remote, con un origin **fetch**, che 'prende' dal server, e un origin **push**, che invia le modifiche e le carica sul server. Per connettere un remote manualmente, si usa **git remote add <nome-del-remote> <url>**. Origin è una convenzione, è il nome predefinito che Git dà al server da cui hai clonato, ma gli si può dare qualsiasi nome.

Non è particolarmente comune né utile, ma un remote può essere un repository. Quindi posso prendere un repository locale e specificarlo come remote. In generale quando si utilizza un repository per fare da server centrale, ma non si usa davvero come repository per lavorarci direttamente, ma solo per prendere le modifiche e condividerle, si può creare un repository che si chiama **bare,** usando **git init --bare,** che crea un repository che non avrà una working directory, ma in quella cartella si troveranno solo i file che normalmente stanno in .git e che di solito rimangono nascosti dall'ls.

Autenticazione
Finché ci si appoggia una cartella in remoto sul nostro file system non ci si deve autenticare. Se però si usa un server remoto, come GitHub, dobbiamo autenticarci in quanto sia che il repository sia pubblica (a meno che non vogliamo che tutti ci possano scrivere) che privata, dovremmo autenticarci per poterla modificare.
La scelta raccomandata è SSH, che si basa su crittografia a chiave pubblica.

1. Chiave privata
    a. Risiede solo nel nostro computer
    b. È segreta e non va mai condivisa
    c. Può essere protetta da un passphrase (per cifrare la password tramite algoritmo a chiave simmetrica)
2. Chiave pubblica
    a. Puoi condividerla liberamente
    b. Va caricata sui servizi su cui vogliamo l'autenticazione via SSH (es GitHub)

Per creare la chiave si usa il comando **ssh-keygen -t ed25519 -C 'tua_email@esempio.com'**. -t è l'algoritmo, mentre –C è un commento per riconoscere la chiave. Il comando chiederà dove salvare i file e di inserire una passphrase. Per gestire più chiavi, si può fare –f ~/.ssh/github_tap che creerà il file github_tap (privata) e github_tap.pub (pubblica).
Per controllare se la chiave funziona, usa **ssh –T [-i <file-della-chiave-privata>] git@github.com**.
Ssh-agent è un servizio che rimane in background, che tiene in memoria le chiavi private sbloccate, così non devi inserire la passphrase ogni volta. Per avviarlo si scrive **eval "$(ssh-agent -s)"**, per aggiungere la chiave all'agent si usa **ssh-add ~/.ssh/github_tap**.
Se si vuole utilizzare chiavi diverse per host diversi, il modo migliore è usare il file di configurazione **~/.ssh/config,** dove si possono specificare i diversi servizi.

Quando siamo connessi con un remote, possiamo fare diverse operazioni:

- Fetch: scarica dal remote tutte le modifiche che non abbiamo ancora, ma non fa automaticamente un merge con la working directory (cioè non cambierà nulla sulla nostra working directory).
- Pull: fa fetch e anche un merge per unire il repository con la mia working directory ('scarica le modifiche che sono sul remote e mettile nella mia working directory'). Quindi è quasi sempre

equivalente a **git fetch origin**, e dopo **git merge origin/main** (sostanzialmente il main sul remoto).

- Pull --rebase: dopo il pull proverà a fare rebase al posto di merge.

Per tenere traccia di quello che succede su remoto uso i tracking branch (o remote-tracking branch), però la differenza è che si possono solo leggere, al contrario dei branch normali su cui si può anche lavorare. Quindi quando noi facciamo fetch si aggiornano questi puntatori per sincornizzarsi su cosa è successo dall'altra parte, però appunto al contrario dei branch normali, questi sono utilizzati solo per tenere traccia.

- Push: spedisce un branch locale su un remoto. La prima volta che si prova a pushare un branch, se dall'altra parte il tracking brench non esiste non va bene e si 'arrabbia'. Quindi la prima volta che si pusha in genere si mette un –u, quindi si scrive git push –u origin mio-branch, e questo crea nel reemoto il tracking branch e associa il ramo dove siamo al tracking branch dall'altra parte. Quindi l'origin/mio-branch è la versione remota del mio mio-branch. Quindi se aggiungo cose in mio-branch e faccio push finiranno anche in origin/mio-branch. Quando si fa push le modifiche si devono poter agganciare: se io ho clonato un certo repository e poi qualcuno ha fatto l'amend dell'ultimo commit, se provo a fare push Git cercherà di agganciarsi ad un commit che però non esiste più (PER QUESTO NON SI DEVE RISCRIVERE LA STORIAAAA!!!).

Nello sviluppo open source, di solito non si ha il permesso di scrivere direttamente sul repository principale. Il flusso di lavoro standard è:

1. Fork: Crei una tua copia personale del repository su GitHub
2. Clone: Cloni il tuo fork sul tuo computer locale
3. Branch: Crei un nuovo branch per la tua modifica
4. Commit & Push: Lavori e fai il push dei commit sul tuo fork (origin)
5. Pull Request (PR): Apri una richiesta formale per integrare le tue modifiche nel progetto originale
6. Code Review & Merge: I manutentori revisionano e, se approvato, uniscono il tuo codice

In questo scenario, ci saranno due remote configurati: origin, ossia il fork su GitHub, creato automaticamente quando cloni da GitHub, e upstream, ossia il repository originale da cui si è fatto il fork. Di solito, per l'upstream si hanno solo permessi di lettura, e serve per tenersi aggiornati. Va aggiunto manualmente con:

git remote add upstream git@github.com:PROGETTO_ORIGINALE/progetto.git

Per aggiornare il main locale con le ultime modifiche del progetto originale:

git fetch upstream
git switch main
git merge upstream/main

Per evitare modifiche accidentali o non autorizzate a rami critici (main, develop), le piattaforme come GitHub e GitLab permettono di impostare delle regole di protezione. Le regole più comuni includono:

- Impedire push diretti: obbliga tutti a passare tramite Pull Request per proporre modifiche
- Richiedere Code Review: impone che almeno una o più persone approvino una Pull Request prima che possa essere unita
- Richiedere il superamento di controlli: blocca il merge se la build, i test automatici o altri controlli di CI/CD falliscono
- Impedire push --force: previene la riscrittura della cronologia del branch
- Impedire la cancellazione del branch

Come far riferimento ad un commit?

I commit sono identificati da un hash, quindi un modo per indetificarli è tramite il loro hash. Non serve usare l'intero hash, bastano quattro caratteri fuorché non sia ambiguo (in progetti piccoli, in più grandi anche sette caratteri per essere sicuri). Si può usare head, che punta al commit attuale.
Per risalire lungo la cronologia, si può usare l'operatore tilde ~. Mentre head è l'ultimo commit, **head~** o **head~1** è quello subito prima (quindi il parent del commit attuale). Si può utilizzare **head~n** per trovare il n-esimo antenato.
Se si è di fronte ad un merge commit, e quindi abbiamo più parent, con tilde ~ va sempre sul primo genitore, mentre se vogliamo scegliere su quale genitore andare indietro nel tempo si può usare **head^1** che è il primo genitore e **head^2** che è il secondo genitore (commit che è stato unito quando si è fatto merge).
Head@{n} permette di navigare il reflog: recupera la posizione che aveva head n movimenti prima. Ogni volta che si sposta head, questa cosa viene scritta nel reflog, che è un log dei riferimenti, che tiene traccia di dove si sposta head. È improbabile che nella vita di tutti i giorni si usi.
É più probabile usare **nome@{data/ora}** (nome di un branch o head), per vedere ad esempio come era il repository ieri (head@{yesterday}).
A volte ci si trova davanti ad una linea di codice e si vuole sapere chi l'ha scritto. Si usa **git blame <nome-file>** che ritorna l'hash del commit, l'autore, la data dell'ultima modifica e il contenuto della linea. Da linea di comando si può anche specificare da che linea a che linea con [-L <da-linea>, <a-linea>].
Rebase può essere usato per tante cose e c'è anche il cosiddetto rebase interattivo, che ti permette di unire più commit in un unico commit più grande, per avere una cronologia più facile da leggere. Si fa con **git rebase –i <commit-precedente-a-quello-dove-iniziare>**: i sta per interattivo e dobbiamo dare a rebase il commit precedente rispetto a quelli che vogliamo rebaseare.
Se volessi prendere un singolo commit e aggiungerlo al branch, **git cherry-pick <riferimento-a-commit>** prende le modifiche introdotte da un singolo commit e le applica come un nuovo commit sul branch corrente.

Docker
Prima di parlare di Docker, è necessario parlare dei namespace all'interno, perché il modo in cui è costruito Docker si basa pesantemente sui namespace di Linux.
I container sono un metodo di virtualizzazione a livello del sistema operativo. Isola completamente un'applicazione da un'altra e dà ad ogni applicazione il suo ambiente con il suo file system, i suoi file di configurazione, ... che non possono andare in conflitto con le altre applicazioni. É quindi un inscatolamento che però, al contrario delle VM, non usa un sistema operativo diverso.
Se io utilizzo una macchina linux e utilizzo i container, posso creare delle specie di 'macchine virtuali' dove però il kernel è quello della macchina host. Quindi le applicazioni che girano in un container sono completamente isolate da altre applicazioni che girano in altri container, e quindi sarà completamente isolata tranne che per il kernel. C'è più limitazione, perché se io ho una macchina Linux non posso fare un container Windows, e non posso nemmeno fare un container con un altro kernel (cioè che se ho un kernel 6.0, non posso fare un test dove uso il kernel 4.0) perché il kernel non cambia. Però su questo kernel posso avere distribuzioni diverse, ad esempio avere Ubuntu, Fedora, Arch, ... e tutte useranno lo stesso kernel ma ognuno avrà il proprio file system, ecc.

L'idea è di andare ad inscatolare un'applicazione con tutto quello di cui ha bisogno, e questa scatola la chiamiamo **immagine**, che è facilmente distribuibile ed eseguibile in modo isolato, in un container. Un

parallelo è immaginare l'immagine come un file di programma .exe in Windows, mentre il container è il processo, cioè il file di esecuzione. Un altro parallelo è che l'immagine è la classe, cioè che descrive come funzionano le cose e poi le istanze di questa classe sono effettivamente i container, da una classe si possono tirare fuori tanti oggetti e ognuno ha il suo stato e vita propria, però condivide con gli altri oggetti della classe i metodi, ecc. Quindi da una singola immagine si possono tirare fuori tanti container. Docker è il primo, il più famoso che è andato a implementare questa idea di container in modo che fosse più facile da utilizzare e l'ha quindi reso popolare. É un ecosistema dove abbiamo una serie di comandi che permettono di creare delle immagini, riusare delle immagini, distribuire facilmente queste immagini e poi eseguire i container. Quando andiamo ad eseguire un container, l'applicazione dentro al container gira in modo ragionevolmente sicuro; possiamo controllare quante risorse utilizza il container.

I namespace sono una caratteristica del kernel Linux: su Windows, Docker Desktop si appoggia a WSL2 (Windows Subsystem for Linux), anche se esistono effettivamente anche dei container Windows che però sono poco utilizzati; su MacOS, Docker si appoggia su una VM Linux. I namespace sono un meccanismo che permette di isolare delle risorse del sistema. Per esempio, un processo, un gruppo di processi, potrebbe non vedere altri processi. Quindi se io vado ad inscatolare un processo dentro ad un namespace, lui vedrà solo i processi che sono dentro a quel namespace e i namespace dentro a quel namespace. Per creare un namespace da riga di comando c'è un comando che si chiama **unshare**, che si basa sull'analoga system call.
Esistono vari tipi di namespace e potrebbero in futuro essere aggiunti altri, i più 'importanti' sono:

- UTS (Unix Time-sharing System), ed è quello che isola il nome dell'host, quindi con un namespace UTS possiamo cambiare il nome dell'host senza cambiare quello fuori
- PID (Process IDentifier), permette ai processi di vedere solo un sottoinsieme di processi; quando un processo dentro ad un namespace chiede quali sono i processi del sistema, vede solo quelli all'interno del suo namespace
- Mount namespace isola i punti di mount
- Network namespce isola parti di rete
- IPC (InterProcess Comunication), permette di isolare semafori e altre varie primitive di sincronizzazione del sistema
- User serve a isolare gli utenti; ad esempio, posso avere un namespace dove c'è root ma è root solo in quel namespace

Se mettiamo tutto assieme, cioè creiamo un namespace con tutto isolato, abbiamo quasi un container alla docker:

```
sudo unshare \
--pid \
--mount \
--uts \
--ipc \
--net \
--user \
--map-root-user \
--mount-proc \
--fork \
/bin/bash
```

Dentro questa shell:

- `hostname <nome>` → cambia l'hostname solo qui (`UTS`)
- `echo $$` → `1` (`PID`)
- `ifconfig -a` → mostra solo l'interfaccia di loopback (`Network`)
- `whoami` → `root` (`User`)
- `mount -t tmpfs none /tmp` → modifica i mount solo qui (`Mount`)

Questa shell è quasi un container "vero", ma ci mancano

- un root filesystem isolato (`chroot`), l'avete visto a SETI
- la gestione della quantità di risorse (`cgroups`)
  - non vedremo i dettagli; l'idea è che i namespace isolano la visione delle risorse, mentre i cgroups ne limitano l'uso (e.s., i processi non posso usare più del 10% della CPU o più di 500MB di memoria)

Le specifiche dell'OCI

L'OCI definisce:

- Image Format Specification, ossia il formato delle immagini, che deve essere strutturata in:
  - Un manifest, ossia un file JSON che descrive i layer dell'immagine
  - Un file di configurazione che specifica come eseguire i container (comandi, variabili d'ambiente, ...)
  - I layer (tarball compressi) che rappresentano il file system (che deve essere immutabile) dei container
- Runtime Specification: descrive come come un 'file system bundle' deve essere eseguito; definisce il formato del file di configurazione (config.json) che specifica i dettagli dell'ambiente in esecuzione. runc è l'implementazione di riferimento di questa specifica
- Distribution Specification che standardizza la comunicazione con i registry/hub di immagini

L'importanza dello standart OCI è che ci rende interoperabili e non ci lega ad una particolare tecnologia: un'immagine costruita con uno strumento OCI-compliant può essere eseguita da qualsiasi altro runtime OCI-compliant (Docker eseguiva di default in root, in quanto un namespace doveva essere creato per forza con root, Podman girava in utente quindi su alcuni aspetti era preferito; ora su Docker si può scegliere come runnare, meglio non root perché se ci sono vulnerabilità ciaone). Stabilità: avere uno standard ben definito e governato dalla community garantisce una base solida e stabile su cui costruire strumenti. Innovazione: standardizzando i componenti base, le aziende possono concentrarsi su altro per rinnovare (es. orchestrazione, ossia gestire container su cui gestire il carico, crea o elimina container in base al demand, ...).

Docker è la piattaforma più popolare per la creazione distribuzione ed esecuzione di container. Si compone di vari pezzi, principalmente in:

- Docker Engine, il motore che esegue il container
- Docker Daemon si usa per parlare, dialogare con il Docker, che implementa le API REST
  - Di default gira come root, ed espone un socket globale /var/run/docker.sock; far girare Doker come root è comodo da una parte, ma se non stiamo attenti rischiamo di mandare in esecuzione dei container con utente root all'interno (può diventare un problema di sicurezza per bug, ecc.)
  - C'è una modalità rootless dove ogni utente ha il suo socket per comunicare con il proprio Daemon
- Docker CLI che utilizziamo per parlare con il Docker Deamon e dargli comandi, e sono comandi da linea di comando

I container si ottengono, nascono, a partire dalle immagini, template immutabili che contengono tutto il necessario per eseguire il software "containerizzato": codice, le dipendenze (librerie, runtime, ecc.), variabili d'ambiente, file di configurazione, ...

Le immagini sono identificate internamente da un hash SHA256, e da **[<namespace>/]<repository>[:<tag>]**, dove namespace identifica un utente/organizzazione (è opzionale, se non lo si mette di default Docker usa 'library'), repository è tipicamente il nome del software e tag ne identifica una particolare versione (default 'latest'). Un esempio può essere 'ubuntu:22.04'.

Allo stesso SHA possono corrispondere tanti nomi diversi. Il modo più semplice per procurarsi un'immagine è quella di scaricarla da un registry, di default Docker Hub, oppure costruite a partire da una 'ricetta' chiamata Dockerfile (vagamente simile a makefile), che contiene una serie di istruzioni che dicono come andare a costruire un'immagine. Vengono scaricate/generate e salvate localmente, quindi è importante fare pulizia perchè non vengono eliminate.
Latest è il default e in teoria è la versione più recente, ma in realtà è una convenzione, è solo un tag.

Comandi principali per gestire le immagini
Intanto, se si sta facendo girare il Docker come root serve **sudo**.

- **[sudo] docker images** (vecchio) o **[sudo] docker image ls** per elencare tutte le immagini
- **[sudo] docker pull [<registry>/]<nome-img>[:<tag>]** per scaricare delle immagini
- **[sudo] docker push [<registry>/]<nome-img>[:<tag>]** per caricare delle immagini per esempio su Docker Hub
- **[sudo] docker rmi <nome-img>[:<tag>]** (vecchio) o **[sudo] docker image rm <nome-img>[:<tag>]** per cancellare l'immagine
- **[sudo] docker image prune [-a]** è il comando che fa pulizia in automatico delle immagini che non sono utilizzate
- **[sudo] docker image tag SRC DST** per taggare un'immagine

Di default, scarica da host docker.io, namespace library, tag latest.

Esempio: scarichiamo prima l'ultima versione di alpine, e poi la versione 3.19, e vediamo che appunto crea due immagini diverse. Però se per esempio scarico l'ultima versione con il tag 'numero' della versione al posto che 'latest', l'image ID sarà uguale ma sono due immagini diverse. Se elimino la versione, cancella più hash perché le immagini in realtà sono formate da una serie di strati e quindi cancella gli hash degli strati.

```
giuliaponassi@MBP-di-Giulia ~ % docker pull alpine
Using default tag: latest
latest: Pulling from library/alpine
55afa1ecc21d: Pull complete
Digest: sha256:28bd5fe8b56d1bd048e5babf5b10710ebe0bae67db86916198a6eec434943f8b
Status: Downloaded newer image for alpine:latest
docker.io/library/alpine:latest

What's Next?
  View a summary of image vulnerabilities and recommendations → docker scout qui
ckview alpine
giuliaponassi@MBP-di-Giulia ~ % docker pull alpine:3.19
3.19: Pulling from library/alpine
17a39c0ba978: Pull complete
Digest: sha256:6baf43584bcb78f2e5847d1de515f23499913ac9f12bdf834811a3145eb11ca1
Status: Downloaded newer image for alpine:3.19
docker.io/library/alpine:3.19

What's Next?
  View a summary of image vulnerabilities and recommendations → docker scout qui
ckview alpine:3.19
giuliaponassi@MBP-di-Giulia ~ % docker image ls
REPOSITORY    TAG       IMAGE ID       CREATED         SIZE
alpine        latest    d529dd0c6e55   7 weeks ago     8.42MB
hello-world   latest    e2ac70e7319a   4 months ago    10.1kB
alpine        3.19      83b2b6703a62   10 months ago   7.4MB
giuliaponassi@MBP-di-Giulia ~ %
```

```
giuliaponassi@MBP-di-Giulia ~ % docker image rm alpine:3.19
Untagged: alpine:3.19
Untagged: alpine@sha256:6baf43584bcb78f2e5847d1de515f23499913ac9f12bdf834811a314
5eb11ca1
Deleted: sha256:83b2b6703a620bf2e001ab57f7adc414d891787b3c59859b1b62909e48dd2242
Deleted: sha256:0b44b2151d78267ab6f2c76208c3be18688f49b2b0afd6852a9533f2cce121c5
giuliaponassi@MBP-di-Giulia ~ % docker image ls
REPOSITORY    TAG       IMAGE ID       CREATED         SIZE
alpine        latest    d529dd0c6e55   7 weeks ago     8.42MB
hello-world   latest    e2ac70e7319a   4 months ago    10.1kB
giuliaponassi@MBP-di-Giulia ~ %
```

Se il nostro utente fa parte del gruppo docker, non serve utilizzare sudo; non è una buona idea per la sicurezza del sistema. Se Docker gira come root, si può condividere una cartella del nostro host dentro al container e se il container gira come root, la home di root, di solito non leggibile, diventa leggibile e scrivibile. L'alternativa sicura è la configurazione rootless.

Un container è l'istanza di un'immagine. Si possono avviare tramite **run** o **start**, fermarli con **stop** o **kill** ed eliminarli con **rm**. Quando mandiamo in esecuzione un container, cioè andiamo a creare un container, quello che dobbiamo dare opzionalmente è il nome del container, altrimenti Docker ne genererà uno automaticamente, e dirgli quale immagine vogliamo runnare. Per vedere quali container sono in

esecuzione posso usare **docker ps**. Per vedere veramente tutti i container, anche quelli che sono terminati, uso **docker ps –a**. Per eliminare un container uso **docker rm <nome-o-id>**, e per rimuovere tutti i container stopped posso fare **docker container prune**. Ad entrambi posso aggiungere **–f** per forzare l'eliminazione. Invece il comando **docker system prune** ripulisce i containe, le immagini, i volumi e le reti.

Alcune opzioni utili quando si lancia il container

- **-u** o **--user <name|uid>[:<group>|<gid>]** permette di specificare con quale nome e gruppo vengono eseguiti i container. Se stiamo usando la modalità rootless, l'utente che fa girare il container è il mio, se invece stiamo usando la modalità dove Docker gira come root, l'utente che gira è quello di root
- **-d** o **--detach** manda in background l'esecuzione del container. In alcuni casi si vuole interagire con il container, ma se invece lancio qualche server che non voglio sia in foreground, posso lasciarlo in background. Se voglio vedere cosa sta succedendo si può usare **docker logs [-f] <nome-o-id>** che fa vedere cosa ha stampato il container nella sua console virtuale
- **--rm** rimuove automaticamente il container quando ha finito l'esecuzione
- Tutte le volte che si ha a che fare con un container interattivo, tipicamente si mette **–it** o **–ti**, e vuol dire che vogliamo la modalità interattiva e comunichiamo al Docker di creare un terminale virtuale per poter interagire con quel container

Quando si lancia un container, all'interno del container gira un solo comando. Alcune volte può essere utile lanciare un secondo comando all'interno dello stesso container, specialmente per fare debugging. Per lanciare all'interno di un container che sta già girando, invece di usare run si usa exec: run parte da un'immagine e crea un nuovo container, mentre exec crea all'interno di un container un nuovo processo. Quindi se per esempio voglio lanciare un'altra shell all'interno dello stesso container che avevo prima, posso specificare il nome del container e poi quale comando voglio lanciare (esempio: docker exec –it pippox busybox sh, che si comporta come una bash, una shell).

Se non specifichiamo nessun user, e in generale non si specifica quando si fa exec, si eredita semplicemente lo user che si è specificato in run.
L'utente 0, ossia l'utente root, dentro al container, viene mappato sull'UID dell'utente: se il mio utente è 1000, quello che è root dentro al container viene visto come UID 1000 nel sistema. Per tutti gli altri ID, viene fatto un remapping che viene descritto dal file /etc/subuid. Se il numero n dell'UID è dentro il container, viene rimappato su il numero s_u+ n – 1, dove s_u è il primo id nel re-mapping di u contenuto in /etc/subuid.

Se si ferma un container, si può far ripartire con **docker start**. L'exec non funzionerebbe perché se un container viene fermato non si può far girare niente al suo interno; non funzionerebbe nemmeno run perché creerebbe un nuovo container.

Networking: cosa succede dentro ai container?

I container di default possono accedere ad internet. I container che girano come root vanno su internet di default. Docker offre varie modalità di network:

- (default) bridge: i container possono vedere gli altri container che sono sulla stessa rete, e se Docker gira come root possono accedere ad internet, mentre se gira in modalità rootless non si hanno i permessi di alterare le regole di iptables e i container potrebbero non riuscire ad accedere ad internet. Dentro ai container, se si ha installato Docker Desktop, si ha come nome simbolico host.docker.internal che corrisponde alla macchina host (mentre 127.0.0.1 o localhost si riferiscono al container stesso e non alla macchina fisica). Se Docker Desktop non l'ha impostato in automatico, si può usare --add-host host.docker.internal:host-gateway. I servizi che vengono aperti dentro al container non vengono visti dalla macchina esterna a meno che non lo si chieda (di default non viene esposta)
- (custom) bridge: come bridge, con DNS server che permette di usare i nomi dei container
- Host: la rete non viene isolata, ma di nuovo in modalità rootless non viene isolata da una roba che è già isolata
- None: esplicitamente non si può andare in rete

È possibile creare le proprie reti, e quindi mettere più container su una loro rete in modo che fra di loro si vedano e possano collaborare. Per vedere le reti si usa il comando **docker network ls**, per crearla si usa **docker network create [--driver bridge] my-network**, per cancellarla uso **docker network rm** e per far partire un container dentro ad una rete, devo dirgli **docker run --network my-network**. Il comando **docker inspect** dà, in formato JSON, delle informazioni su network, container, ecc.

Quando facciamo partire un container, il file system del container è isolato, cioè viene visto solo da quel container, e temporaneo, ossia le modifiche che vengono fatte dentro al container vengono dimenticate quando il container viene buttato via. Ci deve essere un modo per salvare qualcosa, e, su Docker, per permettere di rendere persistenti dei dati, possiamo usare i **volumi**: sono un'area dati che viene creata sulla macchina host e viene montata all'interno del file system di un container. I volumi esistono indipendentemente dai container, quindi se si crea un volume, container diversi, in momenti diversi o anche contemporaneamente, possono montare lo stesso volume, e quindi vedono lo stesso file system e se ci scrivono qualcosa lo vedranno anche gli altri container e soprattutto rimane anche se i container vengono poi buttati via. I volumi sono gestiti da Docker, ma alternativamente si possono montare delle directory tramite quello che si chiama **bind mount** dalla macchina host a dentro il container. Quindi posso dire quando lancio il container la mia directory viene vista come /etc dentro al container, mentre al di fuori rimane la mia directory. Se il container fa modifiche in quella cartella le vedo e rimangono persistenti.
Esiste lo **switch --mount**, che può montare sia i volumi che le directory dell'host, mentre il comando **–v/--volume** può montare directory o volumi di Docker. Mount è quella più esplicita e sopporta più opzioni.

	Volumi	Bind Mount
Gestiti da	Docker	utente
Posizione nel FS host	scelta da Docker	qualsiasi percorso
Portabilità	qualsiasi host con Docker	dipende dalla struttura FS dell'host
Caso d'uso tipico	dati di produzione: db, configurazioni, ...	sviluppo software: codice sorgente, file di configurazione live, ...

Esempi d'uso
Come esempi di uso, useremo Forgejo, ossia un self-hosting git server. Vorremmo avere dei dati persistenti quindi in questo caso andremo a creare due directory: uno per il container di Forgejo e l'altro per Postgres, dove salverà il database in questa directory, mentre Forgejo salverà i dati nell'altro. Per

metterli in comunicazione, creiamo un custom network e questo custom network ha un DNS che permette di usare nomi simbolici per i container in modo di farli parlare. In questa prima fase, faremo 'a mano', cioè creeremo il primo container, creeremo il network, il secondo container, ... Poi vedremo che c'è un modo migliore, ossia compose. Per creare le due directory, facciamo **mkdir –p forgejo-data postgres-data**. Per la rete creiamo un custom bridge con **docker network create forgejo-network**. A questo punto si inizia facendo partire il database, perché Forgejo quando poi lui parte ha bisogno di connettersi al database server (quindi il container Forgejo non può partire se non è partito l'altro container). Per far funzionare PostgreSQL, avviamo il container del database con run:

```
docker run –d \
--name forgejo_db \
--network forgejo-network \
--env POSTGRES_DB = forgejo \
--env POSRGRES_USER = forgejo \
--env POSTGRES_PASSWORD = forgejo \
--mount type = bind, src = ./postgres-data, dst = /var/lib/postgresql/data \
postgres: 14
```

Il –d è per mandarlo in background, diamo un nome al container e lo mettiamo nel network creato prima. Sto lanciando postgres versione 14, e mount mappa una directory del nostro sistema all'interno del volume; src è quella del nostro file system, e dst (destination) sarà dove sarà fatto il bind. La directory locale ./postgress-data diventerà dentro al container /var/lib/postgresql/data. I --env vanno a definire delle variabili d'ambiente.
Non serve usare --user perché l'immagine imposta già un suo utente.
Dalla macchina host non ho modo di parlare con il database, questo perché non ho detto a Docker di esportare nessuna porta. Quindi, se non dico a Docker di collegare una porta da dentro il container a fuori, questo non avviene in automatico.
Non è buona idea passare le variabili d'ambiente sulla riga di comando; questo perché se non si cancella l'history della shell rimane nel file history e quindi qualcuno potrebbe trovare la password del database, e perché se vado a vedere l'elenco dei process con ps, vedo la password del database. Quindi in un ambiente realistico, queste variabili di ambiente verrebbero messe in un file che convenzionalmente si chiama **.env**, e che verrà messo nel .gitignore se si sta usando Git.

Possiamo verificare che il database sia in esecuzione sia con docker ps, che con **docker logs forgejo_db**, che ci fa vedere l'output che ha fatto quel container. Quando si è sicuri che sia attivo, si va ad avviare Forgejo:

```
docker run –d \
--name forgejo \
--network forgejo-network \
-e B_TYPE=postgres -e DB_HOST=forgejo_db:5432 -e DB_NAME=forgejo \
-e DB_USER=forgejo -e DB_PASS=forgejo -v ./forgejo-data:/data \
-v /etc/timezone:/etc/timezone:ro -v /etc/localtime:/etc/localtime:ro \
-e USER_UID=$(id -u) -e USER_GID=$(id -g) \
-p 3000:3000 \
-p 2222:22 \
codeberg.org/forgejo/forgejo:11
```

-e è la versione abbreviata di --env. 5432 è la porta di default di Postgres. Il timezone e localtime vengono messi in read-only (ro). Il –p (abbreviazione di --publish) dice 'voglio che tu colleghi la porta TCP, quella 3000 sull'host con la 3000 dentro al container'. Nel secondo caso, la macchina host sarà 2222 e nel container 22.
Il server SSH (che serve per avere una remote shell) di Forgejo non espone una shell ma è un modo per gestire le operazioni con Git.
Per vedere se funzionano davvero, ci possiamo collegare sulla porta 3000. Se andiamo su **http://127.0.0.1/3000** o su **http://host.docker.internal/3000**, dovrebbe uscire la pagina di installazione di Forgejo, con i campi del database precompilati. Forgejo è esteticamente uguale a GitHub, ed è molto simile anche nell'utilizzo, con repositories, organizations, si possono creare repository nello stesso modo, ...
Per fermare e rimuovere i container e la rete si usa:

docker stop forgejo forgejo_db
docker rm forgejo forgejo_db
docker network rm forgejo-network

Docker Compose
Il Docker Compose è uno strumento che permette di definire e utilizzare applicazioni formate da più container, ossia multi-container. Potrebbe essere utile utilizzare Docker Compose anche per un singolo container, ma è più in situazioni dove si deve configurare più di un container. Docker Compose era inizialmente uno strumento che veniva dato separatamente da Docker, sviluppato da Docker ma non era tra le istanze default di Docker, era un comando chiamato docker-compose ed era un eseguibile diverso da quello di Docker. Adesso Compose è integrato dentro a Docker, quindi quando lo utilizziamo lanciamo il solito comando docker che usiamo per far partire, chiudere container, ecc., usiamo **docker compose**, che va ad utilizzare un file che si chiama **docker-compose.yml,** ossia un file di tipo YAML (yet another markup language), un file di testo con una serie di chiavi valori, dove vado ad elencare quali sono i container che voglio tirare su nella mia applicazione, come si chiama la rete, che volumi voglio mappare, ... Gli argomenti che prima abbiamo scritto 'a mano' su linea di comando vengono messi tutti insieme in questo file yaml.
I comandi di Docker Compose sono **up** per 'tirare su tutto' (avviare), **down** per 'tirare giù tutto' (arrestare) e **build** per costruire l'applicazione. Il nome della directory che contiene il file docker-compose.yml è quello che dà il nome al progetto, cioè l'insieme dei container. Il nome della directory è in un certo senso un namespace del container; questo perché si potrebbe avere lo stesso container in progetti diversi, avere tante istanze, identificabili con il 'prefisso'. Se si vuole usare un altro nome di progetto, è possibile specificare **–p/--project-name.** Di default, se non si mette esplicitamente un container name dentro il Docker Compose, il nome del container è il nome del progetto, il nome del servizio e il un indice: <nome_progetto>_<nome_servizio>_<indice>.

Iniziamo a scrivere un file docker-compose.yml:

```
services:
forgejo:
image: "codeberg.org/forgejo/forgejo:11"
container_name: “forgejo”
        environment:
            - "DB_HOST=db:5432"
```

```
            - "DB_TYPE=postgres"
            - "DB_NAME=pippo"
            - "DB_USER=pippo"
            - "DB_PASS=pippo"
        ports:
            - "3000:3000"
            - "2222:22"
        volumes:
            - "./forgejo-data:/data"
            - "/etc/timezone:/etc/timezone:ro"
            - "/etc/localtime:/etc/local:ro"
        depends_on:
            - db
        networks:
            - forgejo-network
db:
        image: "postgres:14"
        content_name: "forgejo_db"
        environment:
            - "POSTGRES_DB=forgejo"
            - "POSTGRES_USER=forgejo" -
            - "POSTGRES_PASSWORD=forgejo"
        volumes:
            - "./postgres-data:/var/lib/postgresql/data"
        networks:
            - forgejo-network
        restart: unless-stopped
networks:
forgejo-network:
driver: bridge
```

Per avviare l'applicazione Docker Compose, nella stessa directory che contiene docker-compose.yml e le cartelle dei dati, si scrive **docker compose up –d.** Up avvia (o ricrea) i servizi definiti nel docker-compose.yml, mentre –d esegue i servizi in modalità detached. Docker Compose si occuperà di creare la rete forgejo-network, avviare il container forgejo_db per primo ed avviare il container forgejo, assicurandosi che possa connettersi al database. Per verificare che entrambi i container siano in esecuzione usiamo **docker compose ps**. Per fermare e rimuovere tutti i servizi e le reti, si usa **docker compose down**. Down ferma e rimuove i container e le reti. Il 'restart: unless-stopped' è la policy di restart del container, ovvero se il container va in crash, se si vuole che il servizio rimanga su, gli si può dire di restartarlo, e 'unless-stopped' si intende che lo restarta a meno che non sia stato io a stopparlo. Altre opzioni sono no, che è di default, e si intende che se si stoppa si ferma, oppure always e on-failure.

Gli **orchestratori** sono utili quando vogliamo usare più host per garantire il funzionamento dei container anche in caso di guasti, bilanciare il carico usando più host, aumentare/diminuire il numero di container

in base al traffico, ecc. Il più famoso è Kubernetes, ma ci sono tante alternative, e sono degli strumenti in cui si possono dare una serie di regole e loro si adatteranno.

Come 'Dockerizzare' le applicazioni

Finora abbiamo utilizzato immagini già fatte, ma se io ho la mia applicazione e voglio creare un'immagine per essa, come si fa?

Scriviamoci a mano il nostro primo Dockerfile, fatto molto male:

```
FROM mcr.microsoft.com/dotnet/sdk:9.0
WORKDIR /src
COPY . .
ENTRYPOINT ["dotnet", "run"]
```

Questa immagine prende tutto l'ambiente di sviluppo di .NET, si porta dietro i sorgenti che ha in questa directory e lancia il comando donet passando come argomento run. Il dotnet run fa partire l'applicazione, ma se l'applicazione non è ancora stata compilata, la compila e poi la runna.

```
giuliaponassi@MBP-di-Giulia TAP % mkdir toUPPER
giuliaponassi@MBP-di-Giulia TAP % cd toUPPER
giuliaponassi@MBP-di-Giulia toUPPER % dotnet new console -o toUPPER
Creazione del modello "App console" completata.

Elaborazione post-creazione delle azioni in corso...
Ripristino di /Users/giuliaponassi/TAP/toUPPER/toUPPER/toUPPER.csproj:
Ripristino riuscito.


giuliaponassi@MBP-di-Giulia toUPPER % cd toUPPER
giuliaponassi@MBP-di-Giulia toUPPER % ls
Program.cs      obj             toUPPER.csproj
giuliaponassi@MBP-di-Giulia toUPPER % vi Program.cs
giuliaponassi@MBP-di-Giulia toUPPER % vi Dockerfile
giuliaponassi@MBP-di-Giulia toUPPER % ls
Dockerfile      Program.cs      obj             toUPPER.csproj
giuliaponassi@MBP-di-Giulia toUPPER % dotnet run
Si è verificato un errore: Could not find a part of the path '/app/data/input.tx
```

Dopo scriviamo **docker build . -t toupper** per costruire l'immagine. Questa immagine è fatta molto male perché si porta dietro tutto l'ambiente di sviluppo, ogni volta che lo runno ricompila, ...

Non sarebbe furbo che tutte le immagini basate, per esempio, su ubuntu, si portassero dietro tutta la distro. Le immagini sono la "somma" di tanti strati (layer): ogni comando del Dockerfile produce dei layer, che finiscono in un una cache per le build successive. Layer piccoli facilitano il riuso, ma potrebbero "appesantire" il runtime; molte immagini posso usare gli stessi layer. Per migliorare la situazione, è meglio:

- raggruppare i comandi logicamente (più comandi shell in uno stesso RUN)

  ```
  RUN apt-get update && apt-get install -y package1 package2 ... \
  && rm -rf /var/lib/apt/lists/*
  ```

- mettere prima i comandi "stabili", che non cambiano, e poi quelli che verranno più facilmente modificati
- usare build multistage

In un multistage build, al posto di avere un unico FROM, abbiamo più FROM. Il primo pezzo sarà la parte di build, mentre il secondo pezzo sarà effettivamente il runtime.

```
# Stage 1: Build
FROM mcr.microsoft.com/dotnet/sdk:9.0 AS build
WORKDIR /src
COPY . .
RUN dotnet publish -c Release -o /app/publish
```

Stage 2: Runtime
FROM mcr.microsoft.com/dotnet/runtime:9.0
WORKDIR /app COPY --from=build /app/publish .
RUN mkdir /app/data
ENTRYPOINT ["dotnet", "toUPPER.dll"]

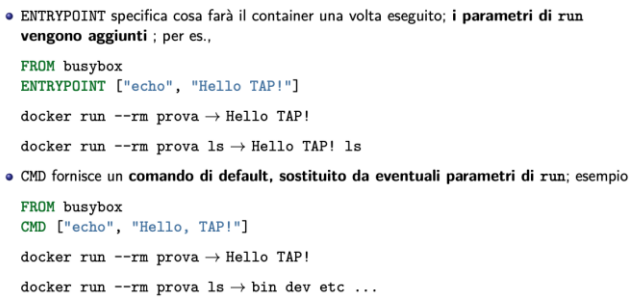
Nella prima parte vado a compilare e 'pubblicare' nella directory /app/publish la versione compilata del mio programma. Dopodiché l'immagine vera che vado a creare, fa il COPY, prendendo dall'immagine che ha creato in build, la directory /app/publish e la metti nella directory corrente di lavoro. In questo modo, nell'entrypoint chiede di eseguire direttamente la DLL che è il risultato della compilazione dello stage 1.

```
giuliaponassi@MBP-di-Giulia toUPPER % docker run -v ./Program.cs:/app/data/input
.txt:ro toupper
Contenuto del file in maiuscolo:
CONST STRING FILEPATH = "/APP/DATA/INPUT.TXT";
TRY {
        STRING CONTENT = FILE.READALLTEXT(FILEPATH);
        STRING PROCESSEDCONTENT = CONTENT.TOUPPER();
        CONSOLE.WRITELINE("CONTENUTO DEL FILE IN MAIUSCOLO:");
        CONSOLE.WRITELINE(PROCESSEDCONTENT);
} CATCH (EXCEPTION EX) {
        CONSOLE.WRITELINE($"SI È VERIFICATO UN ERRORE: {EX.MESSAGE}");
}

giuliaponassi@MBP-di-Giulia toUPPER %
```

Migliorando il Dockerfile, vediamo anche che la nuova immagine si è ridotta parecchio di dimensione:

```
REPOSITORY    TAG       IMAGE ID       CREATED              SIZE
toupper       latest    48bb3d45c589   About a minute ago   210MB
<none>        <none>    3896c94c3f92   30 minutes ago       851MB
```



Nell'immagine creata, abbiamo usato ENTRYPOINT, che dice qual è il comando che viene eseguito dall'immagine quando si fa partire il container. Oltre ad ENTRYPOINT, si può usare anche CMD, e hanno un significato leggermente diverso. Quando si usa ENTRYPOINT, quando si tira su un container, se si specificano degli argomenti, questi vengono aggiunti a quelli che erano già stati messi. Con CMD, se non ci sono argomenti, esegue il comando normalmente, mentre se si specificano degli argomenti, quelli vanno a sostituire quelli che c'erano.
Se dentro ad un Docker Compose si specifica un comando di build, quando si tirano su i servizi potete dirgli di fare la build. Se non si chiede esplicitamente, se non trova l'immagine la crea usando Dockerfile. Con --build si forza di rifare la build.

Per condividere le immagini, usiamo **docker push** per caricarle su un registro remoto. Con **docker pull** scarica un'immagine dal registro; teoricamente non serve perché quando si fa run, se Docker non ha l'immagine la scarica in automatico.

C#
C# è nato come una 'copia' di Java, sono rivali e si copiano a vicenda lol.
C# è solo uno dei tanti linguaggi che può girare su .NET. Ce ne sono altri di Microsoft come F#, VB.NET, ... ma in generale .NET è stato proprio pensato per far girare linguaggi diversi. I linguaggi

diversi interagiscono tramite il **Common Language Runtime** (CLR), che è la piattaforma di esecuzione, l'equivalente della JVM (ma CLR è più efficiente perché JVM è stata progettata principalmente per Java). Gestisce l'esecuzione del codice, compilato in quello che si chiama CIL (Common Intermediate Language) o MSIL (Microsoft Intermediate Language), comunque un linguaggio intermedio, Gestisce la memoria, la sicurezza e la compilazione JIT (Just-In-Time).
Come fanno linguaggi diversi ad interpellare il .NET? Esiste il Common Type System (CTS), ossia un insieme di regole che valgono per tutti i linguaggi che sono implementati su .NET. Quindi, ad esempio, tutti i linguaggi su .NET che vogliono usare un intero a 32bit utilizzeranno lo stesso tipo 'system.int32', anche se in C# si usa int ed è la stessa cosa. Infatti, i tipi primitivi di C# sono solo 'alias' per i tipi del framework, che distingue fra **tipi valore** e **tipi riferimento.**

Keyword C#	Tipo CLR (Namespace System)	Tipo .NET	Allocazione
`int`	`System.Int32`	Valore (`struct`)	Stack/Inline
`double`	`System.Double`	Valore (`struct`)	Stack/Inline
`bool`	`System.Boolean`	Valore (`struct`)	Stack/Inline
`char`	`System.Char`	Valore (`struct`)	Stack/Inline
`long`	`System.Int64`	Valore (`struct`)	Stack/Inline
`string`	`System.String`	Riferimento (`class`)	Heap
`object`	`System.Object`	Riferimento (`class`)	Heap

String e object sono la stessa cosa, possono essere usati in maniera intercambiabile. I tipi valore sono quelli che Java chiama 'tipi primitivi': int, char, double, ... In una variabile dichiarata di tipo valore, la variabile contiene il valore, cioè che se si dichiara una variabile intera, il suo contenuto è proprio il numero. Nei tipi riferimento invece il contenuto nella variabile non è un oggetto, ma il riferimento, cioè il nome, il puntatore, all'oggetto, come fa Java con i tipi classe e i tipi interfaccia. C# permette di definire nuovi tipi valore; si possono definire delle classi, analoghe alle classi di Java, ma anche le struct che invece sono dei tipi valore. Se dichiaro una struct che ha due interi come campi, quando si dichiara una variabile di quel tipo, nello spazio della variabile ci saranno due interi. Se invece si dichiara una classe con due campi, nello spazio della variabile c'è sempre il riferimento e i due interi dovranno essere allocati sull'heap facendo una new.

Dove vengono allocati i dati? Ci sono due spazi principali (non ci sono solo questi ma questi due sono i più importanti; i campi statici non stanno in nessuno dei due):

- Lo **stack** è una struttura dati last in first out (pila), allocare qualcosa nello stack consiste nell'aggiungere un pezzo; è un puntatore che va su e giù e basta. Velocissimo dal punto di vista di allocazione e deallocazione perché si deve solo 'spostare il puntatore'. Viene usato per le variabili locali delle funzioni e per altri metadati che servono durante l'esecuzione. Per esempio, per ricordarsi dove deve tornare un'istruzione record: quando si chiama una funzione il sistema deve ricordarsi dove si è interrotto per poi riprendere l'esecuzione da lì quando si fa record, e deve allocare lo spazio per le variabili locali della funzione. Lo stack è molto utile da questo punto di vista perché le variabili locali di una funzione, se la funzione non sta girando, non ci sono; se si ha una funzione ricorsiva che chiama sé stessa, ad esempio, sette volte, si avranno sette spazi per allocare quelle variabili. Ogni attivazione di una funzione ha le sue variabili. Ogni volta che si va a chiamare una funzione, il sistema crea uno stack chiamato 'frame di attivazione', che non è altro che lo spazio per i metadati per ricordarsi dove si era interrotto per la chiamata, e lo spazio per allocare le variabili.
- Ci sono dei valori che non devono essere distrutti quando si esce dalla funzione: tutto ciò che non deve essere distrutto al ritorno dalla funzione, viene messo nello **heap**. L'heap è una zona di memoria a cui chiedere un pezzo di memoria, che deve essere in una zona libera dello heap.

Quindi la sua allocazione è molto più costosa, in quanto, quando si chiede la memoria all'heap, il sistema deve andare a guardare cosa c'è di allocato, ... La deallocazione, invece, è gestita dal sistema, tramite Garbage Collector (GC): quando un oggetto che è stato allocato non è più raggiungibile, allora il Garbage Collector può liberare quello spazio, che sarà di nuovo disponibile da utilizzare per l'allocazione.

Tipi Valore

Una variabile di tipo valore è una variabile che contiene direttamente i suoi dati. I tipi valori sono int, double, bool, e quelli definiti dal programmatore, che si possono definire tramite struct. Quando vengono allocati, se sono delle variabili locali o dei parametri vengono allocati sullo stack; ma se fanno parte di un oggetto, non sono allocati sullo stack, ma stanno dentro a quell'oggetto. Quando si assegna un tipo valore, viene fatta una copia intera del valore perché la variabile contiene tutta la roba e poi se ho due variabili separate, tutto il contenuto di uno viene copiato sull'altro. Tutti i tipi valore ereditano implicitamente da un tipo che si chiama System.ValueType, e abbastanza ragionevolmente non possono essere null (null è un riferimento nullo, quindi non sono riferimenti); hanno sempre un valore di default che è lo 0 corrispondente, quindi int e double 0, bool false, ...
Un tipo enumerativo (enum) è un tipo valore che consiste in un insieme di costanti nominali.

```
public enum Season {
    Spring,   // 0
    Summer,   // 1
    Autumn,   // 2
    Winter    // 3
}
Season s = Season.Autumn;
int value = (int)s; // value è 2
Season s2 = (Season)0; // s2 è Season.Spring
```

Un altro modo per definire dei tipi valore sono le struct. In C# le definizioni di classi, strutture, ... devono stare dopo il codice(?).

```
Point p1 = new Point { X = 10, Y = 20 };
Point p2 = p1;
p2.X = 100;
Console.WriteLine($"p1.X={p1.X}"); // Output: ?

public struct Point { public double X; public double Y; }
```

Prendendo questo codice come esempio, se Point è una **public struct**, l'output sarà 10, perché p1 e p2 sono due punti diversi della stessa struct, quindi abbiamo p1 = {10, 20} e p2 = {100, 20}.
Se invece Point è una **public class**, p1 e p2 sono due nomi diversi dello stesso oggetto, come se fossero due puntatori che puntano alla stessa cosa. Quindi, quando si cambia la X di p2, cambierà pure quella di p1 perché riferiscono allo stesso oggetto.
Questa è la grossa differenza tra una variabile di tipo valore, che inscatola il valore, e la variabile di tipo riferimento, che fa riferimento a qualcosa che sta da un'altra parte (heap).

Capita di interfacciarsi con dei database, e quando succede nel linguaggio object oriented si hanno delle classi che corrispondono alle tabelle dei database. Nei database i campi possono essere nullabili o non nullabili. Si può avere un campo intero nullabile, cioè che sul database può contenere il valore null oppure un intero. Nel file mapping questa cosa può creare problemi, perchè se ho una variabile, un campo, di tipo intero, non posso segnargli un valore. Allora in C# hanno aggiunto il tipo **nullable<T>**, o meglio la scorciatoia **T?.** Quindi un tipo valore seguito da un ? vuol dire che può essere null.

Tipi Riferimento

I tipi riferimento, invece di contenere il valore stesso, contengono un riferimento ('puntatore', anche se non c'è aritmetica dei puntatori) ai dati. Tutte le volte che vengono allocati dei valori di tipo riferimento, loro vengono sempre allocati sullo heap. Invece i riferimenti possono essere sullo stack (esempio di prima), oppure essere in-line, ossia all'interno di un altro oggetto sullo heap (esempio: ho una classe con un campo che è di tipo classe, fa riferimento ad un altro oggetto, e così via). Quando si assegnano due variabili di tipo riferimento, quello che viene copiato da una parte all'altra è solo riferimento, avendo una situazione di **aliasing**, ovvero più variabili che puntano e fanno riferimento allo stesso oggetto sullo heap. Storicamente, le variabili di tipo riferimento potevano essere null, ma da un po' si è deciso di mettere di default non nullabili, e solo se vogliamo rendere esplicitamente null possiamo mettere un ? Come nei database.
Scriviamo **new** per inizializzare una variabile **var**, che dichiara una variabile locale del tipo dell'espressione di inizializzazione. **required**, indica che un campo deve essere necessariamente inizializzato (dagli inizializzatori nell'espressione new o da un costruttore annotato con [SetsRequiredMembers]).
Un **tipo delegate** è un tipo riferimento che fa riferimento a un metodo (e non un oggetto). É l'equivalente C# di un puntatore a funzione in C. Se prendiamo come esempio MathOperation op (che può essere sum, mul, ...), op prende due interi, ci fa qualcosa e restituisce un intero.

```
MathOperation op = Sum;
int res1 = op(10, 5);
op = Mul;
int res2 = op(10, 5);
Console.WriteLine($"res1={res1}, res2={res2}"); // Output: ?

static int Sum(int x, int y) { return x + y; }
static int Mul(int x, int y) { return x * y; }

public delegate int MathOperation(int a, int b);
```

C# moderno usa pesantemente anche la programmazione funzionale, quindi capita spesso di fare dei metodi che prendono in input delle funzioni, restituiscono delle funzioni, chiamano delle funzioni, ... Ci sono dei 'tipi di sistema' generici che rappresentano tutte le funzioni void che prendono n argomenti e tutte le funzioni che restituiscono un certo tipo prendendo n argomenti:

- Action<T1, T2, ...> corrisponde a un metodo void che prende arogmenti di tipo T1, T2, ...
- Func<T1, T2, ..., TRes> corrisponde ad un metodo che prende argomenti di tipo T1, T2, ... e restituisce un valore di ritorno di tipo TRes
- Predicate<T> è sostanzialmente equivalente a Func<T, bool> ma non c'è conversione implicita in nessuna delle due direzioni. Prima era l'unico, ora è stato 'sostituito' (anche se esiste ancora) da Action e Func

Espressioni Lambda

Le espressioni Lambda sono una sintassi concisa per creare delle istanze di tipi delegate (e anche di expression tree ma ne parliamo dopo).

```
MathOperation op_custom = (a, b) => a + b;
Func<int, int, int> op_func = (a, b) => a + b;
Action<string> hello = (name) => {
    Console.WriteLine($"Ciao, {name}");
    Console.WriteLine(op_custom(1,2));
    Console.WriteLine(op_func(1,2));
};

hello("Mario");

public delegate int MathOperation(int a, int b);
```

Una closure si verifica quando una lambda cattura una variabile definita nel suo scope esterno (nell'esempio, lo scope esterno è ciò che è dichiarato fuori action. Se io dentro ad action scrivo 'op_custom = (a, b) => 33' al posto di ritornare a+b, ritornerà 33). Il compilatore crea una classe dove

mette le variabili catturate, in modo tale che tutte le assegnazioni vengano allocate sullo heap e gestite dal Garbage Collector.

```
int factor = 10;
Func<int, int> multiply = (n) => n * factor;
int res1 = multiply(5);
factor = 20;
int res2 = multiply(5);
Console.WriteLine($"res1={res1}, res2={res2}");
```

Cosa succede se catturiamo una variabile in un for loop?

```
var actions = new List<Action>();
for (int i = 0; i < 3; i++)
    actions.Add(() => Console.WriteLine(i));

foreach (var action in actions)
    action();
```

In questo esempio, si inizializza la variabile actions; var in C# è un modo per non scrivere esplicitamente il nome di una variabile, ma lasciare che il compilatore se lo scelga da solo, come con il tipo statico dell'espressione di inizializzazione.
Il risultato sarà 3 3 3 perché viene catturata la variabile int i: esiste un'unica copia di int i che assume i valori 0, 1, 2 e 3. Quando raggiunge 3, il i < 3 diventa falso e quindi usciamo dal for. Quando nel foreach stamperemo actions(), stamperemo 3 volte il valore corrente di i, ossia 3. Se avessimo voluto prendere il valore di i all'esecuzione del for, avremmo dovuto farne una copia.

```
var actions = new List<Action>();
for (int i = 0; i < 3; i++) {
    int copy = i;
    actions.Add(() => Console.WriteLine(copy));
}
foreach (var action in actions)
    action();
```

Da C# 5, con il foreach non serve fare la copia, perché è il compilatore stesso che, quando si scrive un foreach, dietro le quinte lo espande, dove ad ogni giro viene creata una variabile item che corrisponde all'elemento corrente: quindi se si cattura item si cattura la copia corrente.

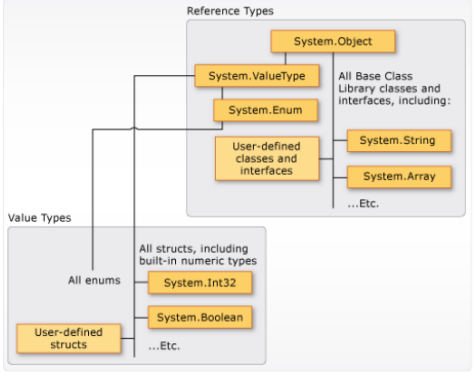
```
foreach (int i in new List<int> {0, 1, 2})
    actions.Add(() => Console.WriteLine(i));
```

diventa qualcosa tipo:

```
var elems = new List<int> { 0, 1, 2 };
for (IEnumerator<int> ie=elems.GetEnumerator(); ie.MoveNext();) {
    var item = ie.Current;
    actions.Add(() => Console.WriteLine(item));
}
```

Sistema di Tipi Unificato

In C# c'è un sistema di tipi unificato, in cui tutti i tipi (int, bool, double, ecc.) derivano direttamente da object. Quindi abbiamo il gruppo dei reference type.



Abbiamo il gruppo dei reference type. Tecnicamente System.ValueType è il 'padre' di tutti i tipi valori, ed è un tipo riferimento, mentre tutti quelli che derivano da lui sono value types. Poi abbiamo la parte dei reference type, quindi string, object, ecc. e tutte le cose che definiamo noi come utenti, quindi le classi, le interfacce, i delegate, … Questo vuol dire che su un int si può chiamare membri di object: si ha un supertipo comune Object, quindi anche int ha metodi.

```
int a = 10;
string s = a.ToString();
```

Adesso nel linguaggio e nella piattaforma ci sono i tipi generici, quindi da un certo punto di vista alcuni feature diventano quasi superflue; un tempo però non c'erano i tipi generici. Se si voleva fare una collezione, una lista, di qualcosa, il modo migliore per approssimarlo, si faceva il **boxing**, la conversione implicita che permette di convertire un valore (di tipo valore) in un riferimento (di tipo riferimento), inscatolandolo: il CLR alloca la memoria sullo heap, il valore viene copiato a questa nuova locazione e viene restituito un riferimento a questo nuovo 'box' sullo heap.
L'**unboxing**, ossia l'operazione inversa, consiste nel prendere un tipo valore che è stato inscatolato da un'operazione di boxing e riportarlo ad essere tipo valore. Mentre il boxing è un'operazione implicita, ossia che non serve nessun cast, se si fa l'assegnazione di tipo valore a tipo riferimento, il compilatore in automatico inserisce il boxing, mentre per il contrario, ossia l'unboxing, serve un cast. Questo perché mentre è sempre possibile prendere un tipo valore e inscatolarlo mettendolo sull'heap, viceversa, se ho object che punta a qualcosa, ma non so a cosa, devo mettere un'operazione esplicita che chiede di estrarre il tipo 'giusto', nel senso che il compilatore non può garantirci che l'object punti al tipo che vogliamo estrarre, quindi il cast a runtime verificherà che, se ad esempio gli avevo chiesto di prendere un intero, che quel riferimento sia davvero un'intero.

I tipi generici in C# possono essere stanziati su qualsiasi tipo, non solo sui tipi reference, e quindi si può fare ad esempio una lista di int, cosa che in Java non si può fare. Questo ha delle ripercussioni non banali sulle prestazioni: l'allocazione sullo heap è molto più costosa del salvare qualcosa su una variabile locale. Se io tutte le volte devo che mettere un qualcosa in una collezione, tirarlo fuori, devo allocarlo, sto perdendo del tempo (e spazio). Mentre CLR (common language runtime), quindi la piattaforma di .NET, quando sono stati introdotti i tipi generici nel linguaggio è stata aggiornata, in Java questo non è stato possibile (ragioni contrattuali?).

Il problema del dei (Billion-Dollar Mistake)
Il null, ossia il riferimento nullo, è stato fonte di moltissimi bug nella storia dello sviluppo software. Tony Hoare, la persona che ha introdotto il concetto, l'ha poi classificato come 'my billion dollar mistake'. Mentre in C# inizialmente e fino alla versione 7, i riferimenti erano annullabili (come in Java), ma le persone si sono rese conto che sono più i casi in cui ci aspettiamo che punti a qualcosa rispetto ai casi in cui non punta a nulla. Una nuova feature del compilatore, dal C# 8, cambia le regole: i nullable reference type. Di default adesso se io scrivo il nome di un tipo reference, sto 'promettendo' di restituire un oggetto di quel tipo, e non restituirò mai null. Se invece un metodo può in alcuni casi restituire null, ci mettiamo un punto interrogativo **?**. Quindi se c'è un reference + punto interrogativo, è un reference nullabile; se non lo metto, il compilatore controlla che il riferimento sia del tipo corretto e che non sia null. Per abilitare questa caratteristica, si deve avere una dichiarazione nel file di progetto, che viene messa di default quando si crea un progetto: **<Nullable>enable</Nullable>**. Se non si vuole questa caratteristica, si può non mettere ma comporta dei warning ecc. Ci sono anche degli operatori che permettono di scrivere codice molto compatto e controlla se un valore è nullo o non è nullo: si usa il

doppio punto interrogativo **??**, oppure doppio punto interrogativo e uguale **??=**, che è sostanzialmente la stessa cosa ma nell'assegnazione.
Altre cose che possono essere comode sono il punto interrogativo + punto **?.** e il punto interrogativo quadra **?[]**. Il primo (?.) dice 'se questo riferimento sta puntando effettivamente a qualcosa, allora accedo al riferimento, faccio il metodo ecc., altrimenti null, non è niente il riferimento e continua ad essere niente. Nel secondo (?[ ]) controllo un array che non so se è nullo o meno: se c'è un array accedo ad un indice, altrimenti il risultato è null. Esiste l'operatore punto esclamativo ! che sostanzialmente dice al compilatore che sicuramente il metodo che lo precede non sarà null.

Classi e interfacce
Le classi e le interfacce sono sostanzialmente uguali, e cambia solo un po' la sintassi: sia in Java che in C# l'ereditarietà per le classi è l'ereditarietà singola, mentre per le interfacce l'ereditarietà è implementare. Java usa **extense** per specificare la superclasse e usa **implements** per dare la lista delle interfacce che si vogliono implementare. C# usa la stessa sintassi di C++, usando i due punti : , ma per C# c'è il vincolo che il primo nome che si mette è la superclasse e poi ci saranno le interfacce. In generale in .NET tutti i nomi di interfaccia iniziano con una I (i maiuscola): quindi se qualcosa ha un nome che inizia con I, è un'interfaccia.

Entrambi i linguaggi supportano l'ereditarietà singola da una classe base e multipla dalle interfacce. Cambia solo la sintassi.

Java usa `extends` e `implements`

```
public class Manager
  extends Employee
  implements IBonus, IReport {
    // ...
}
```

C# usa i due punti (`:`) per tutto; la classe base deve venire prima delle interfacce

```
public class Manager
  : Employee, IBonus, IReport {
    // ...
}
```

Per quanto riguarda le cosiddette **proprietà**, C# permette di definirle: le proprietà sono cose che sembrano dei metodi, ma su cui si possono mettere dei controlli. Accedono ad un campo, ma quando si accede viene chiamato il getter corrispondente, poi si scrive dentro a questo campo chiamando set. In Java devo esplicitamente mettere un **get name** che è un metodo e **set name** come altro metodo. In C# posso dichiarare ad esempio string name, e poi mettere get/set. Quel name posso leggerlo e scriverlo come se fosse un campo, ma posso eseguire il codice per verificare ad esempio se il metodo può essere implementato (codice get e codice set con degli if e altri controlli).

Java l'interfaccia definisce i metodi `getX` e `setX`.

```
public interface IPerson {
    String getName();
    void setName(String name);

    int getAge(); // Sola lettura
}
```

C# l'interfaccia definisce la proprietà stessa, specificando quali metodi di accesso (`get`, `set`) devono essere implementati

```
public interface IPerson {
    string Name { get; set; }

    int Age { get; } // Sola lettura
}
```

Come Java 8+, C# 8.0 ha introdotto le implementazioni di default nelle interfacce: questo permette di aggiungere dei membri nelle interfacce senza rompere la compatibilità con le classi esistenti che la implementano.
I costruttori chiamano i costruttori della classe base: in Java si usa **super(...)**, mentre in C# si usa **base(...)**. Per usare un altro costruttore della stessa classe, si usa **this(...)**.
Sia Java che C# distinguono tra due tipi di uguaglianza:

- Uguaglianza per riferimento (identità): due variabili puntano allo stesso oggetto in memoria sull'heap. È il comportamento di default dell'operatore **==** per i tipi riferimento
- Equivalenza per valore: due oggetti (che possono essere anche differenti in memoria) contengono lo stesso dato, rappresentano la stessa cosa. É l'obiettivo dei metodi **equals( )** e **Equals( )**. In C# come convenzione i metodi hanno il primo carattere maiuscolo

Uguaglianza tipi valore in C#

- Operatore ==: non è definito per le struct definite dal programma, e può essere definito manualmente.
- Metodo Equals(object obj): Funziona, ma è lento. L'implementazione di default (**System.ValueType**) usa **reflection** per confrontare i campi uno per uno, però questo causa boxing: la struct deve essere "inscatolata" sullo heap per essere passata come object.
- Consigli: Se C# 10+ si può usare **record struct**, che fa tutto in automatico. C'è la possibilità di implementare l'interfaccia **IEquatable<T>** (es. IEquatable<Point>): così fornisce un metodo **Equals(T other)** type-safe che non causa boxing, perchè non prende più object come in Equals.

Tutte le volte che si va a ridefinire come si fa l'uguaglianza (tramite l'override), si deve anche ridefinire l'HashCode, che in Java si chiama **hashCode( )** e in C# si chiama **GetHashCode( )**, questo perché c'è un **Contratto** che dice che se due oggetti sono uguali per equals( )/Equals( ), devono avere lo stesso hash.

Java

```
@Override
public boolean equals(Object obj)
{ ... }

@Override
public int hashCode()
{ ... }
```

C#

```
public override bool Equals
                    (object? obj)
{ ... }

public override int GetHashCode()
{ ... }
```

Un'altra cosa che C# permette di specificare sono le conversioni di tipo. La conversione implicita (implicit) viene usata per conversioni "sicure" (senza perdita di dati o eccezioni), e il compilatore le esegue automaticamente. La conversione esplicita (explicit) viene usata per conversioni "pericolose" (possibile perdita di dati o eccezioni): richiede un cast esplicito (tipo) da parte del programmatore.

Sia C# che Java usano i modificatori di accesso per controllare la visibilità di classi e membri. In C# abbiamo: public, private, protected, internal, protected internal, e private protected. In Java abbiamo public, private, protected, e "default" (AKA package-private, non è una keyword).
Questi tre modificatori sono concettualmente identici

- public: nessuna restrizione
- private: solo all'interno della classe che lo definisce
- protected: all'interno della classe che lo definisce e delle classi che la estendono; per Java, anche le classi nello stesso package

In Java un confine è il **package**, mentre in C# il "confine" è **l'assembly**, modificatore **internal**, che è la DLL compilata, indipendentemente dal namespace (analogo di package per Java).

- **protected internal** significa **protected** O **internal**, quindi le classi figlie e tutto ciò che è nello stesso assembly
- **private protected** (introdotto in C# 7.2) significa **protected** E **internal**, quindi solo che classi figlie che si trovano nello stesso assembly